DESMITIFICANDO EL BACKEND



7 ABRIL

Introducción a la Programación y Computación 2

Creado por: Tobías Francisco Zamora Santos

Tabla de contenido

1

Construcción y Documentación de APIs RESTful con .NET y Swagger 1

¿Qué es una API? 1

¿Qué significa REST? 1

¿Qué es .NET (ASP.NET Core)? 3

¿Qué es Swagger (y OpenAPI)? 3

OpenAPI y el "Contrato de la API" 3

¿Qué es un Contrato de API? 4

La gran ventaja en el mundo real 4

La magia de .NET 5

Creación del Proyecto y Dependencias Iniciales 6

Creación del Proyecto en .NET 6

1. Selección del Proyecto en la interfaz de Visual Studio 6

2. Selección de Nombre y Ubicación 7

3. Selección de opciones finales 8

4. Sintaxis para una terminal 9

Instalación de Dependencías: Agregando Swagger al proyecto 10

Opción 1: Desde la Interfaz Gráfica (Visual Studio) 10

Opción 2: Tip de Experto (Desde la Terminal) 10

Limpieza y Configuración en Program.cs 11

Visualización de Swagger 12

Limpieza de la Plantilla (Adiós, WeatherForecast) 13

Arquitectura en Capas y la Inyección de Dependencias (Para el ejemplo de una Tienda) 14

Arquitectura del Proyecto: Separación de Responsabilidades 14

Inyección de Dependencias y Ciclos de Vida 15

Creación de Modelos: Definiendo las Entidades 16

1. Clase Producto.cs 17

2. Clase Usuario.cs 18
3. Clase LoginRequest.cs 18
Capa de Repositorios: Nuestra Bóveda de Datos en Memoria 19
Registro en Program.cs: ¿Por qué usar Singleton? 20
La Capa de Servicios: El Cerebro de la Operación 22
Creando nuestro UsuarioService.cs 22
Creando el ProductoService.cs 25
Los Controladores: Exponiendo nuestra API al mundo 28
1. El Arte del Enrutamiento (Routing) en .NET 28
El Comodín [controller] 28
El Misterio de la Virgulilla ~ (Tilde) 28
Agrupación en Swagger con [Tags] 29
2. Entendiendo IActionResult y los Codigos de Estado 29
Familia de los 200: "Todo salió bien" 30
Familia de los 400: "El Cliente se equivocó" 30
Familia de los 500: "El Servidor falló" 30
Creando el ProductosController.cs 31
Creando el UsuarioController.cs 34
Desmitificando Swagger: La Magia de la Reflexión en .NET 36
1. El escaneo de los Controladores 37
2. Análisis de métodos y parámetros 37
3. Disección del Modelo (Aquí ocurre la magia) 37
4. Traducción a JSON Schema 37
JSON para Probar los Endpoints desde Swagger 38
Crear Productos 38
Listar Productos 40
Crear Usuario 41
Listar Usuarios 44
Seguridad con JWT y Pruebas en Postman 44

Instalación de la Dependencia 45
1. El Mito de la “Encriptación” y la Clave Secreta 45
2. El Ciclo de Vida del Token: Dos Momentos Clave 46
Momento 1: El Login (Aquí CREAMOS la firma) 46
Momento 2: Usar las rutas protegidas (Aquí VERIFICAMOS la firma) 47
3. ¿Qué se debe (y qué no se debe) poner en el Payload? 47
4. ¿Por qué usamos Postman y qué es un “Barere Token”? 48
5. Modificaciones en el Código 49
Paso A: Configurar Program.cs 49
Paso B: Actualizar UsuarioService.cs 52
Paso C: Actualizar UsuariosController.cs 52
Paso D: Probar los enpoints en Postman 54

Construcción y Documentación de APIs RESTful con .NET y Swagger

Una **API RESTful** es una forma de diseñar servicios en internet para que diferentes sistemas (apps, páginas web, servidores) puedan comunicarse entre sí usando HTTP de manera organizada.

¿Qué es una API?

Una **API (Application Programming Interface)** es básicamente un "puente" que permite que un programa le pida información o acciones a otro.

Ejemplo simple:

- Tu app del clima → pide datos a un servidor → recibe temperatura

¿Qué significa REST?

REST (Representational State Transfer) es un estilo de arquitectura que define **cómo debe funcionar esa comunicación**.

Una API RESTful sigue estas reglas:

1. Usa HTTP
Trabaja con métodos conocidos:

- **GET** → obtener datos
- **POST** → crear datos
- **PUT** → actualizar datos
- **DELETE** → eliminar datos

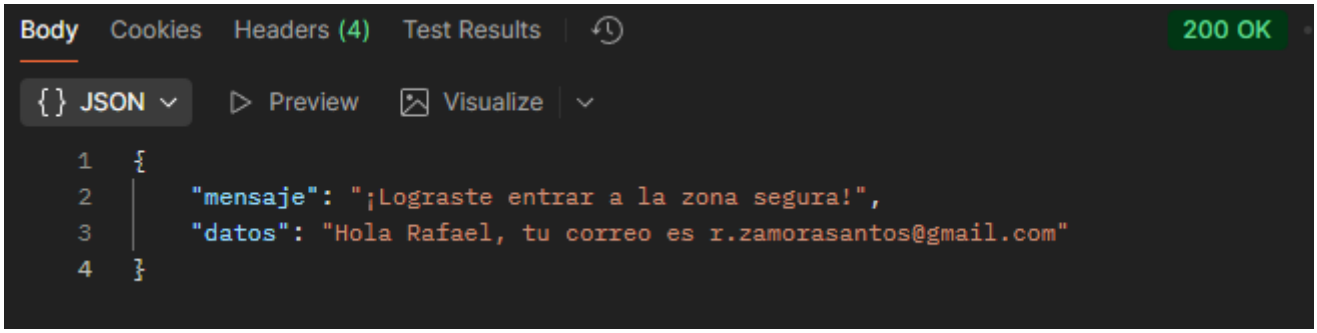
2. Todo son recursos (URLs)
Cada cosa tiene una dirección (endpoint):

- /usuarios → lista de usuarios
- /usuarios/1 → usuario específico

3. Es sin estado (stateless)

Cada petición es independiente.
El servidor **no recuerda** lo que hiciste antes.

4. Usa formatos estándar
Normalmente responde en **JSON**:



Un ejemplo real sería:
Una app de tienda online donde el frontend en React o una app móvil le pregunta a tu API:

- *"Dame todos los productos"* → GET /productos
- *"Agrega este producto al carrito"* → POST /carrito
- *"El usuario pagó, elimina la orden"* → DELETE /orden/12

REST vs RESTful

- **REST** → es el **concepto / estilo arquitectónico**
- **RESTful** → es el **adjetivo** (algo que sigue REST)

Ejemplo claro

- ❌ "Tengo una API REST" → puede ser ambiguo
- ☑ "Tengo una API RESTful" → significa que:
  - Usa bien **GET, POST, PUT, DELETE**
  - Tiene rutas bien estructuradas (/usuarios/1)
  - Es **stateless**
  - Usa formatos estándar (JSON)

¿Qué es .NET (ASP.NET Core)?

ASP.NET Core es el framework (entorno de trabajo) de código abierto creado por Microsoft que estamos utilizando para construir nuestra API.

- **La analogía:** Si construir una API fuera construir una casa, el lenguaje C# son los ladrillos y .NET es toda la maquinaria pesada, los andamios y los planos arquitectónicos que te permiten levantar la casa rápido y de forma segura.
- **¿Por qué es importante hoy?** Atrás quedaron los días en que Microsoft era solo para Windows. El .NET moderno es **multiplataforma** (puedes programar y desplegar tu API en Windows, Mac o servidores Linux) y es uno de los frameworks más rápidos y demandados a nivel empresarial en todo el mundo. Nos provee de herramientas listas para usar, como el enrutador de URLs y la inyección de dependencias.

¿Qué es Swagger (y OpenAPI)?

Swagger es una herramienta visual que genera documentación interactiva para nuestra API de forma completamente automática. Trabaja bajo el estándar universal llamado **OpenAPI**.

- **La analogía:** Imagina que tu API es la cocina de un restaurante cerrado. El equipo de Frontend (los meseros) no puede entrar a ver qué hay. **Swagger es el menú del restaurante.** Le dice exactamente al Frontend qué platillos existen (endpoints), qué ingredientes llevan (modelos/JSON) y qué van a recibir a cambio (códigos de estado).
- **¿Por qué es mágico?** En lugar de escribir un documento de Word larguísimo explicando cómo usar tu API, Swagger lee tu código C# usando "Reflexión" (analiza tus controladores y modelos) y dibuja una página web bonita. Lo mejor de todo es que no solo es un manual de lectura, sino que permite hacer pruebas reales en el navegador con un solo clic, sin necesidad de programar ni una sola línea de código en el Frontend.

OpenAPI y el “Contrato de la API”

A menudo la gente usa las palabras *Swagger* y *OpenAPI* como si fueran lo mismo, pero en realidad son dos cosas distintas:

- **OpenAPI** es el **estándar** (las reglas y el vocabulario).

- **Swagger** es la **herramienta** (la página web visual que dibujó la interfaz usando esas reglas).

Para entender OpenAPI, tenemos que hablar del concepto más importante en la comunicación de software: **El Contrato de la API**.

¿Qué es un Contrato de API?

Imagina que, en tu proyecto de fin de semestre, un equipo está haciendo el Frontend (la página web en React o Angular) y tú estás haciendo el Backend en .NET. Ambos equipos necesitan empezar a trabajar el mismo día, pero... ¿cómo sabe el equipo de Frontend qué datos enviarte si tú aún no has terminado de programar el código?
Aquí es donde entra el Contrato.
OpenAPI es una forma de escribir un documento (un "contrato") que establece reglas estrictas e inquebrantables sobre cómo se van a comunicar las dos partes. Este contrato define:

1. **Las Rutas:** ¿A qué URLs se puede llamar? (ej. /api/Usuario/crear).
2. **Los Verbos:** ¿Qué método HTTP se usa? (GET, POST, PUT, DELETE).
3. **El Formato de Entrada (Request):** "Si quieres crear un producto, me *debes* enviar un objeto JSON que tenga obligatoriamente un nombre (texto) y un precio (número). Si mandas el precio como texto, te rechazo la petición".
4. **El Formato de Salida (Response):** "Si todo sale bien, te prometo devolver un código 200 OK. Si el producto ya existe, te prometo devolver un 400 Bad Request con un mensaje de error".

La gran ventaja en el mundo real

En las empresas profesionales, este contrato (que por debajo es un archivo de texto llamado swagger.json) se diseña y se "firma" el Día 1.

- El equipo de Frontend toma el contrato y construye todas sus pantallas simulando las respuestas.
- El equipo de Backend toma el contrato y programa toda la lógica y la base de datos.
- Al final del mes, cuando conectan ambas partes, todo encaja a la perfección como piezas de Lego, porque ambos obedecieron el mismo contrato.

La magia de .NET

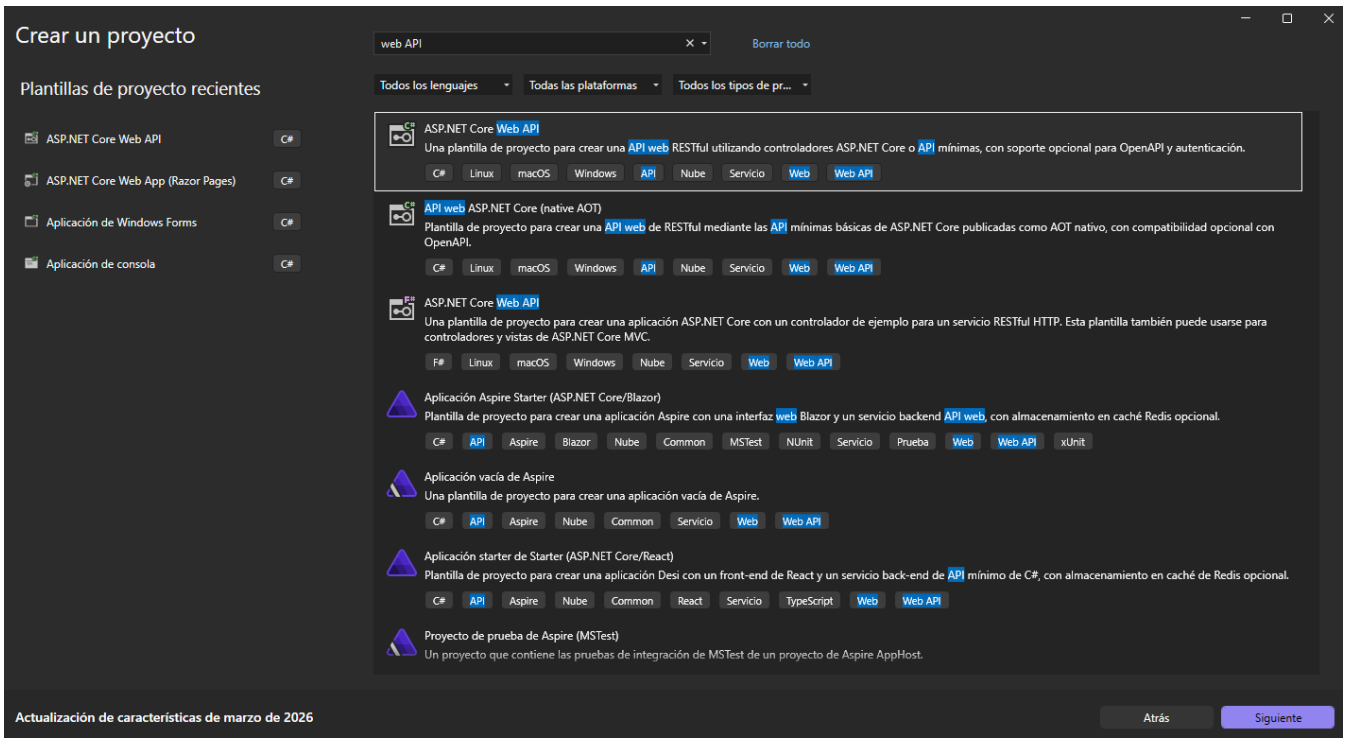
En otros lenguajes, a veces tienes que escribir este gigantesco contrato a mano, línea por línea. Sin embargo, en nuestro proyecto de .NET, logramos hacer **"Código primero" (Code-first).** Al poner los atributos [ApiController], [HttpGet] y definir nuestras clases Usuario y Producto, .NET y Swashbuckle leyeron nuestro código y **redactaron el contrato OpenAPI automáticamente por nosotros.**

Creación del Proyecto y Dependencias Iniciales

Creación del Proyecto en .NET

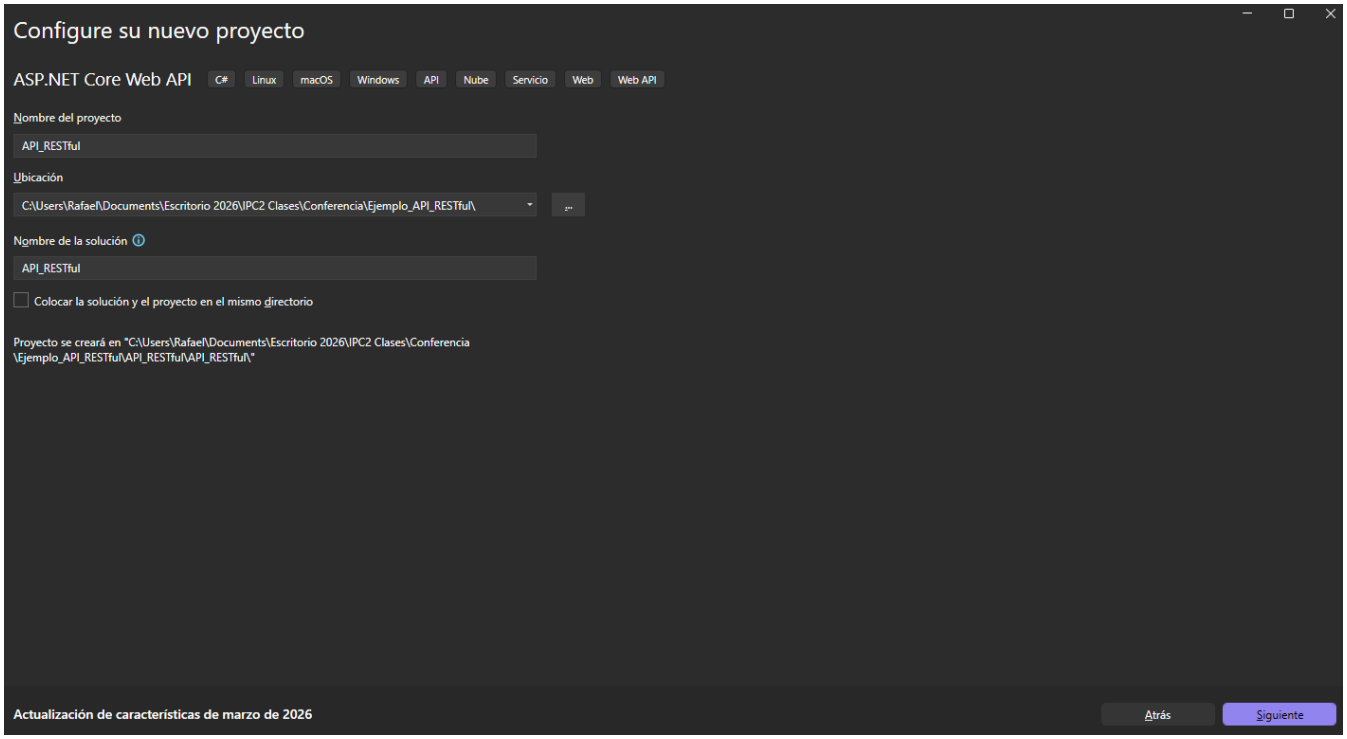
Para estas alturas, el estudiante ya tiene que tener instalado el .NET SDk, un IDE como lo seria Visual Studio o un editor de texto como lo seria Visual Studio Code.

1. Selección del Proyecto en la interfaz de Visual Studio



Se selecciona el tipo de proyecto, **ASP.NET Core Web API**, tomar en cuenta que arriba a la izquierda aparezca **C#** ya que es el lenguaje que se usara.

2. Selección de Nombre y Ubicación



Se le coloca un nombre al proyecto y se selecciona la ubicación de este, además de eso en la pantalla de creación, Visual Studio nos presenta la opción: **"Colocar la solución y el proyecto en el mismo directorio"**. Para decidir qué hacer, primero debemos entender dos conceptos clave de .NET:

- **La Solución (.sln):** Es el contenedor principal (la caja grande).
- **El Proyecto (.csproj):** Es la aplicación o módulo de código en sí (lo que va dentro de la caja).

Opción A: Dejar la casilla DESMARCADA (Recomendado - Buenas Prácticas) Es el estándar en la industria. Al dejarla desmarcada, Visual Studio crea una carpeta principal para la Solución y, dentro de ella, una subcarpeta exclusiva para tu API.

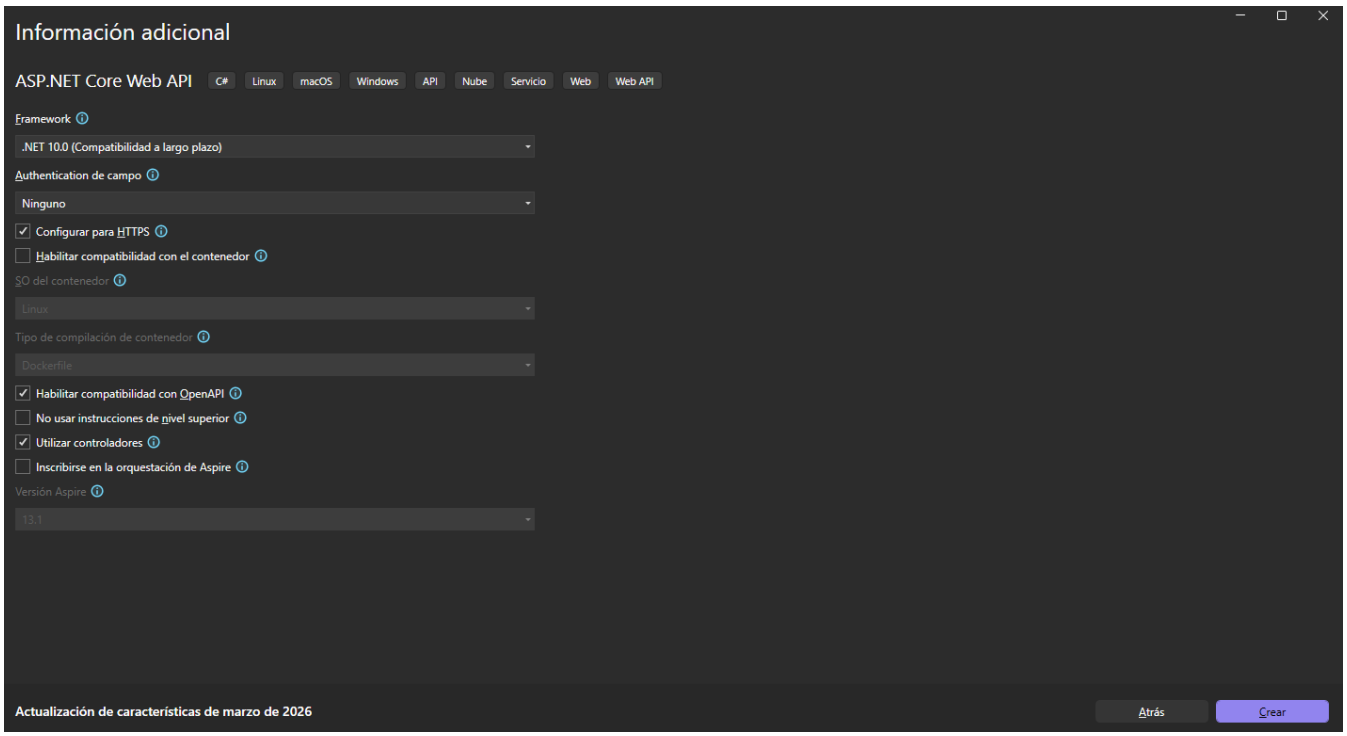
- **¿Por qué usarlo?** Está pensado para escalar. Si mañana quieres agregar al mismo repositorio un proyecto de Pruebas Unitarias (Unit Tests), o aislar tus Modelos en una Biblioteca de Clases, cada cosa tendrá su propia carpeta limpia dentro de la Solución principal.

Opción B: MARCAR la casilla Si la marcas, el archivo de la Solución (.sln) y el del Proyecto (.csproj) se guardan revueltos exactamente en la misma carpeta raíz.

- **¿Por qué usarlo?** Únicamente para proyectos "juguete", pruebas de concepto muy rápidas o tareas donde estás 100% seguro de que el sistema constará de una sola pieza de código y nunca va a crecer.

Es mejor dejarla desmarcada, para que se acostumbren a la estructura de carpetas.

3. Selección de opciones finales



En la siguiente ventana, Visual Studio nos preguntará qué características queremos incluir en nuestra API base. Para este proyecto, dejaremos las opciones tal como se muestran en la imagen, destacando los siguientes puntos clave:

- **Framework (.NET 10.0):** Nos aseguramos de usar la versión LTS (Soporte a Largo Plazo) más reciente para tener las últimas mejoras de rendimiento y seguridad.
- **Configurar para HTTPS:** Lo dejamos **marcado**. Es el estándar de seguridad actual de la web, garantizando que la comunicación viaje cifrada desde el primer día.
- **Habilitar compatibilidad con OpenAPI:** ¡Vital dejarlo **marcado**! Esta es la opción que le indica a Visual Studio que instale las librerías necesarias para generar nuestro "Contrato" y dibujar la interfaz de Swagger automáticamente.
- **Utilizar controladores:** Lo dejamos **marcado**. Como explicamos en la arquitectura, esto nos creará la carpeta Controllers por defecto, permitiéndonos

separar las responsabilidades de forma profesional en lugar de tener todo amontonado en un solo archivo.

- **No usar instrucciones de nivel superior (Top-level statements):** Lo dejamos **DESMARCADO**.
    - *Nota importante:* Al dejar esta casilla en blanco, le estamos diciendo a C# que queremos usar su **sintaxis moderna**. Esto elimina todo el código repetitivo antiguo (namespace, internal class Program, static void Main(string[] args)) y nos deja un archivo Program.cs increíblemente limpio, minimalista y fácil de leer, donde solo colocamos las instrucciones que realmente importan.

4. Sintaxis para una terminal

El comando para crear el mismo proyecto desde una terminal sería el siguiente:

dotnet new webapi -n API_RESTful -f net10.0 --use-controllers

¿Qué significa cada parte de este comando?

- **dotnet new webapi**: Le dice a .NET que use la plantilla base para una API.
- **-n API_RESTful**: Es el nombre de nuestro proyecto (se creará una carpeta con este nombre automáticamente).
- **-f net10.0**: Especificamos que queremos usar la versión 10.0 del framework.
- **--use-controllers**: ¡Este es el parámetro más importante! Por defecto, las versiones nuevas de .NET intentan crear algo llamado "Minimal APIs" (todo el código en un solo archivo). Al agregar esta bandera, obligamos a .NET a usar la arquitectura limpia de Controladores que vimos en esta guía.

Nota: *Las características como HTTPS, OpenAPI y las instrucciones de nivel superior (Top-level statements) ya vienen activadas por defecto al usar esta plantilla en la terminal.*

Si alguien pregunta como ser crea la solución, se tiene lo siguiente:

Crea la solución vacía

dotnet new sln -n API_RESTful

Crea el proyecto (en su propia subcarpeta)

dotnet new webapi -n API_RESTful --use-controllers

Mete el Proyecto dentro de la solución

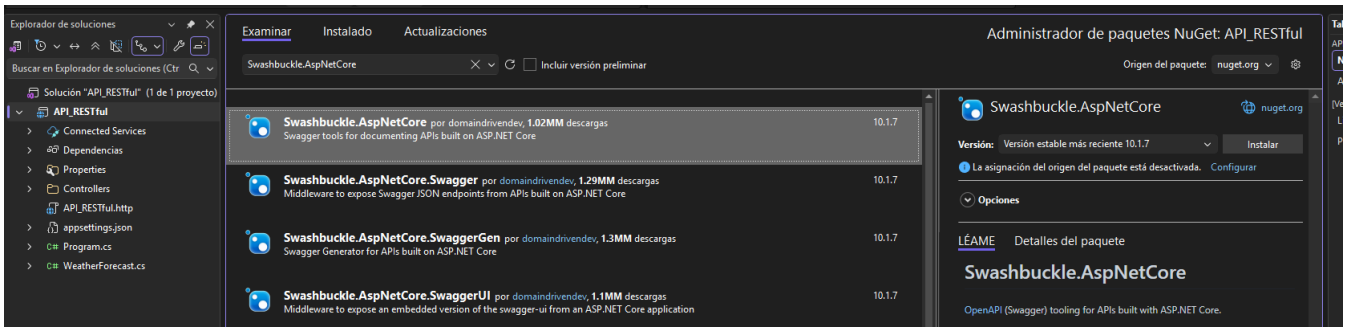
dotnet sln add API_RESTful/API_RESTful.csproj

Instalación de Dependencías: Agregando Swagger al proyecto

Aunque al crear el proyecto le indicamos a Visual Studio que queríamos soporte para OpenAPI, nosotros vamos a utilizar la herramienta líder de la industria para visualizar este contrato: **Swagger**.
Para integrarlo, necesitamos instalar un "paquete" (una librería de código escrita por la comunidad). En el ecosistema de .NET, estos paquetes se manejan a través de **NuGet**.

Opción 1: Desde la Interfaz Gráfica (Visual Studio)

1. Ve al menú superior y haz clic en **Herramientas > Administrador de paquetes NuGet > Administrar paquetes NuGet para la solución**.
2. Haz clic en la pestaña **Examinar**.
3. En el buscador, escribe: Swashbuckle.AspNetCore.
4. Selecciónalo, marca la casilla de tu proyecto a la derecha y haz clic en **Instalar**.



Opción 2: Tip de Experto (Desde la Terminal)

Si estás usando la terminal o Visual Studio Code, solo debes navegar a la carpeta de tu proyecto y ejecutar este comando:

dotnet add package Swashbuckle.AspNetCore

Limpieza y Configuración en Program.cs

Una vez instalado el paquete, debemos decirle a nuestra API que lo utilice. Toda esta configuración inicial ocurre en un solo archivo: el **Program.cs**.
Como instalamos Swashbuckle (que ya trae su propio motor completo para manejar OpenAPI), vamos a limpiar el código nativo que .NET nos generó por defecto para evitar choques de versiones.

Abre tu archivo Program.cs, **elimina** las líneas que dicen **builder.Services.AddOpenApi();** y **app.MapOpenApi();**, y reemplázalas con la configuración de Swagger.

El archivo debe quedar así de limpio:

De esto:



```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
builder.Services.AddOpenApi();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();

```

A esto:



```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
//builder.Services.AddOpenApi();
// Agregamos el generador de Swagger (Reemplaza a AddOpenApi)
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    //app.MapOpenApi();
    // Activamos la interfaz gráfica de Swagger solo si estamos en modo Desarrollo
    app.UseSwagger();
    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();
```

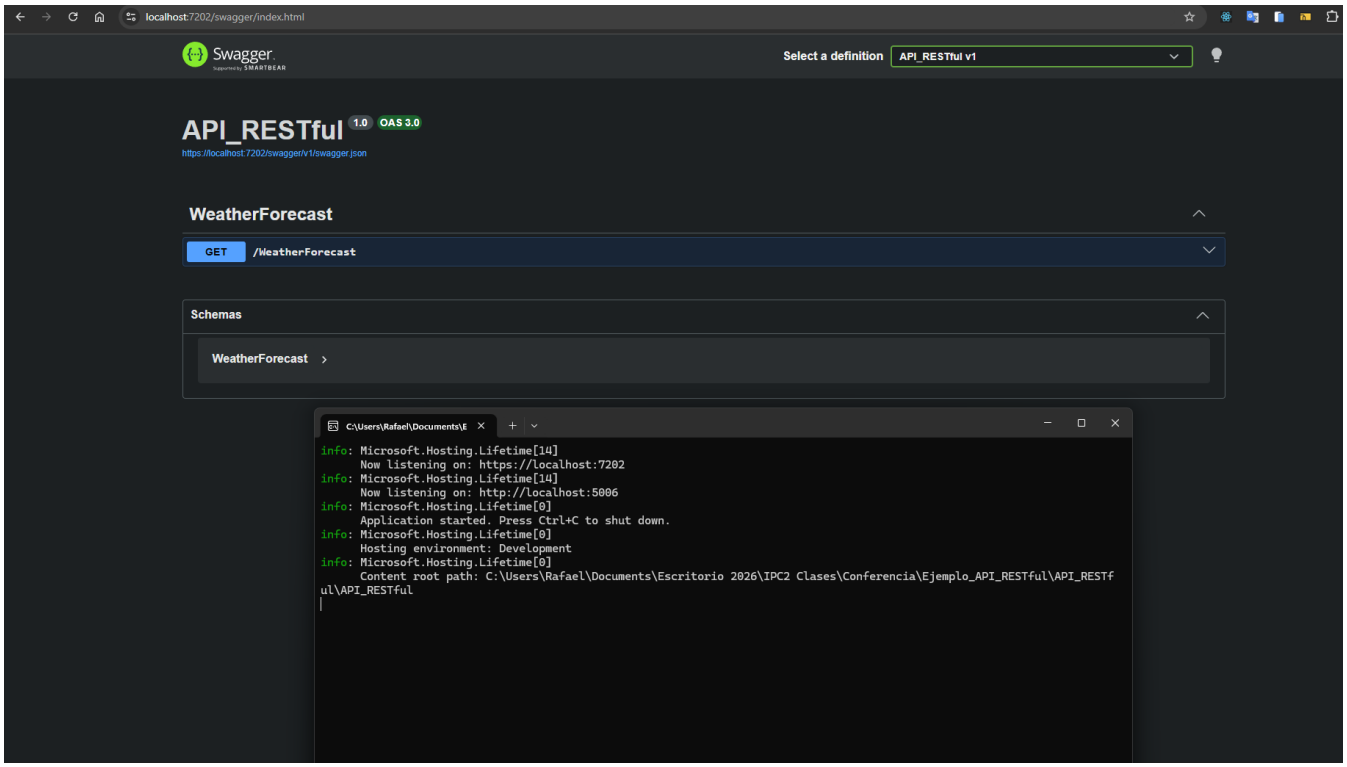
Visualización de Swagger

Una vez configurado nuestro **Program.cs**, es hora de encender el motor. Al ejecutar el proyecto (presionando el botón "Play" en Visual Studio o usando el comando **dotnet run** en la terminal), se abrirá una consola negra indicando que nuestro servidor local está escuchando peticiones.

Para ver la magia en acción:

1. Revisa en la consola la dirección web donde se levantó tu servidor (por ejemplo, https://localhost:7202).

2. Abre tu navegador web, pega esa dirección y **agrega /swagger al final** de la URL (ej. https://localhost:7202/swagger).



¡Felicidades! Lo que estás viendo en pantalla es la interfaz gráfica de tu contrato OpenAPI. Swagger ha dibujado dinámicamente una página web donde podrás interactuar con tu API sin necesidad de programar un Frontend.

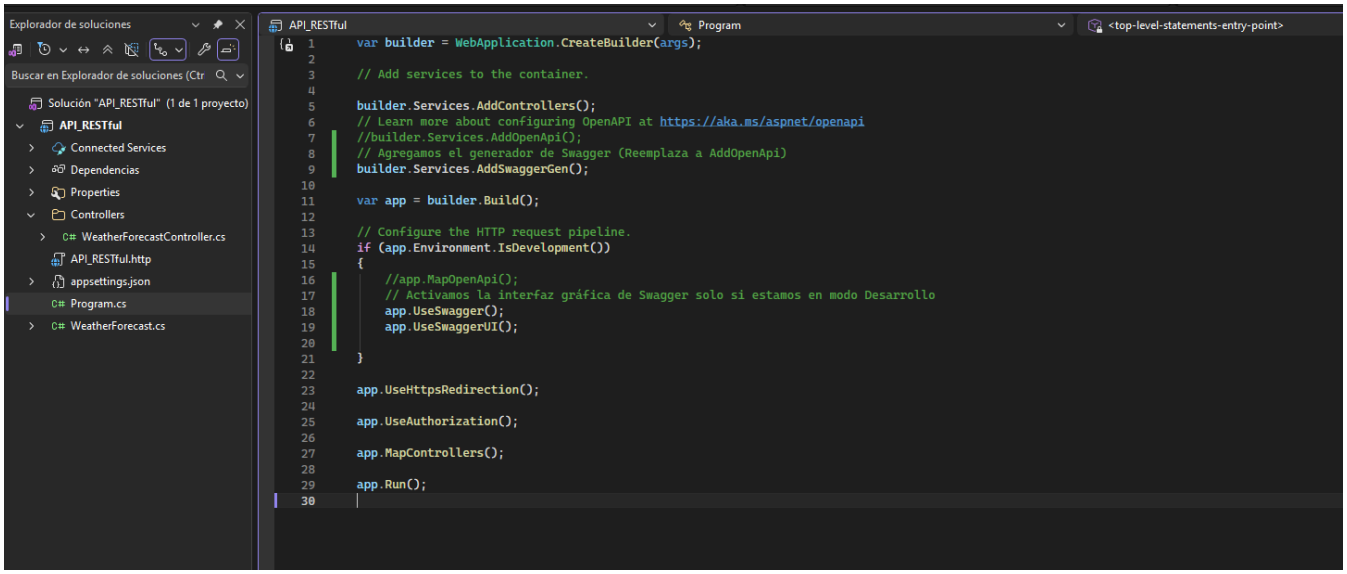
Limpieza de la Plantilla (Adiós, WeatherForecast)

Al entrar a Swagger, ya existe un controlador y un esquema llamado **WeatherForecast** (Pronóstico del clima). No hay de que preocuparse, no escribiste eso por accidente. **¿Qué es esto?** Es simplemente un código de ejemplo que la plantilla de .NET genera automáticamente para demostrarte que la arquitectura básica funciona.

Limpieza del Proyecto (Housekeeping): Para la conferencia, no vamos a dar el pronóstico del clima; vamos a construir una tienda real. Por lo tanto, la mejor práctica antes de empezar a programar es dejar nuestro proyecto como un lienzo en blanco. Ir al **Explorador de soluciones** y **eliminar** sin miedo estos dos archivos de ejemplo:

1. El archivo **WeatherForecastController.cs** (ubicado dentro de la carpeta Controllers).

2. El archivo **WeatherForecast.cs** (ubicado en la raíz del proyecto).



Arquitectura en Capas y la Inyección de Dependencias (Para el ejemplo de una Tienda)

Arquitectura del Proyecto: Separación de Responsabilidades

Para que nuestra API sea escalable, fácil de leer y de mantener, no vamos a escribir todo el código en un solo lugar. Vamos a dividir nuestro proyecto en capas, donde cada carpeta tendrá una única responsabilidad.

En la raíz de nuestro proyecto, crearemos la siguiente estructura de carpetas:

1. **Models (Los Datos):** Aquí vivirá la estructura de nuestra información. Son clases simples (conocidas como POCOs) que solo contienen propiedades. Por ejemplo, cómo se ve un Usuario o un Producto. No tienen lógica, son solo los "moldes".
2. **Repositories (La Bóveda):** Es la capa de acceso a datos. Es la **única** parte de toda la aplicación autorizada para leer, guardar o modificar registros. En nuestro

ejemplo simularemos una base de datos usando listas en memoria, pero en la vida real, aquí es donde irían las consultas SQL o el código de Entity Framework.

3. **Services (El Cerebro):** Aquí reside toda la **Lógica de Negocio**. ¿El usuario es mayor de edad? ¿El producto que quiere comprar realmente existe en el catálogo? El Servicio hace todas estas validaciones puras de C# antes de pedirle al Repositorio que guarde algo.
4. **Controllers (Los Recepcionistas):** Como vimos, esta carpeta ya viene creada. Su único trabajo es "hablar HTTP". Reciben la petición JSON del cliente (el Frontend o Swagger), se la pasan al Servicio para que trabaje, y devuelven un código de estado web (como un 200 OK o un 400 Bad Request). ¡Los controladores no toman decisiones de negocio!

Inyección de Dependencias y Ciclos de Vida

Para que los Controladores puedan hablar con los Servicios, y los Servicios con el Repositorio, .NET utiliza un patrón de diseño llamado **Inyección de Dependencias**. En lugar de crear los objetos a mano con la palabra new, le pedimos a .NET que nos los provea automáticamente.
Para configurar esto en nuestro **Program.cs**, debemos definir el "Ciclo de Vida" de cada pieza. Usaremos dos tipos principales:

1. Patrón Singleton (AddSingleton)

- **Qué significa:** Se crea **una sola instancia** de la clase cuando la aplicación arranca, y esa misma instancia se comparte para absolutamente todas las peticiones de todos los usuarios.
- **Dónde lo usaremos:** En nuestro **Repositorio**.
- **¿Por qué?:** Como estamos usando listas en memoria para simular nuestra base de datos, necesitamos que esas listas persistan. Si no fuera Singleton, cada vez que hiciéramos un clic, la lista se reiniciaría y perderíamos a todos los usuarios creados. Al ser Singleton, garantizamos una "única fuente de la verdad" global.

2. Patrón Scoped (AddScoped)

- **Qué significa:** Se crea una instancia nueva **por cada petición HTTP**.
- **Dónde lo usaremos:** En nuestros **Servicios**.
- **¿Por qué?:** Es la mejor práctica de seguridad y rendimiento. Si 50 alumnos envían un **POST** al mismo tiempo para crear su usuario, .NET crea 50 instancias

independientes del **UsuarioService**. Cada petición tiene su propio espacio de trabajo aislado, asegurando que la validación de los datos de un alumno no se cruce ni interfiera por accidente con la de otro. Al terminar de procesar la petición, el servicio se destruye para liberar memoria RAM.

Nota Arquitectónica: ¿Por qué no usamos Entity Framework?
Si has visto tutoriales de .NET avanzado, probablemente hayas escuchado sobre **Entity Framework Core (EF Core)**, el ORM oficial de Microsoft que conecta nuestras clases con una base de datos real (como SQL Server, PostgreSQL o MySQL).
Para esta conferencia, **no utilizaremos EF Core ni una base de datos tradicional**. ¿Por qué?

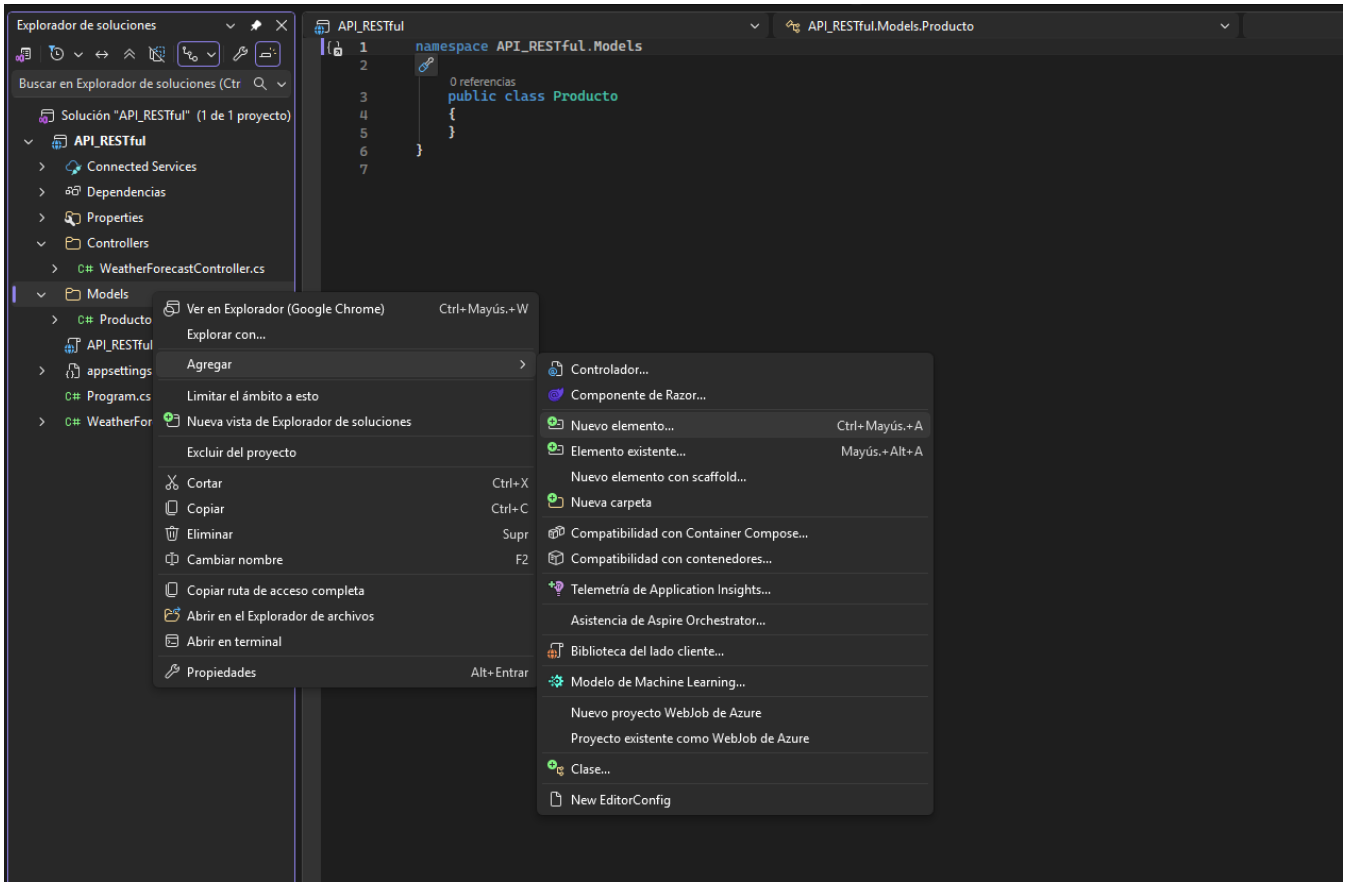
1. **Foco en el objetivo:** El objetivo de hoy es desmitificar la capa web (Swagger, Controladores, JSON, Seguridad JWT). Configurar motores de bases de datos, cadenas de conexión y migraciones añadiría mucha "ruido" y complejidad innecesaria a la demostración.
2. **Aislamiento del aprendizaje:** Queremos entender cómo viaja la información desde el Frontend hasta el Backend.
3. **Nuestra Solución (El Mock):** En lugar de una base de datos real, utilizaremos **Listas en Memoria** (List<Usuario>) combinadas con el patrón **Singleton**. Esto simulará a la perfección el comportamiento de una base de datos (podremos guardar, leer y buscar datos) pero viviendo temporalmente en la memoria RAM de nuestra computadora mientras la API esté encendida.

En el mundo profesional, la ventaja de usar la Arquitectura en Capas es que el día de mañana puedes borrar nuestra clase Repositorio en memoria, conectar Entity Framework, ¡y los Controladores y Servicios ni siquiera se darán cuenta del cambio!

Creación de Modelos: Definiendo las Entidades

En la carpeta **Models**, definiremos la estructura de la información que nuestra tienda va a manejar. Estas clases sirven como el "lenguaje común" entre el Cliente (Frontend) y el Servidor (Backend).

Tanto para **Carpetas** como para **Clases**, para crearlas se debe de dar click derecho sobre la ubicación y añadirlas normalmente.



1. Clase Producto.cs

Representa los artículos que venderemos. Es la entidad más básica de nuestro sistema.

```cs
namespace API_RESTful.Models
{
    public class Producto
    {
        public int Id { get; set; }
        public string Nombre { get; set; } = string.Empty;
        public decimal Precio { get; set; }
        public string Resena { get; set; } = string.Empty;
    }
}
```

2. Clase Usuario.cs

Es nuestra entidad principal. Nota cómo incluimos una lista de productos para simular una relación entre el usuario y sus compras.

```
namespace API_RESTful.Models
{
    public class Usuario
    {
        public int Id { get; set; }
        public string Nombre { get; set; } = string.Empty;
        public string Correo { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty; // ¡Recuerda: nunca devolver esto en el token!

        // Relación: Un usuario puede tener muchos productos
        // Pueder ser null si el usuario no tiene productos asociados,
        // por eso el tipo es List<Producto>? y se inicializa con una nueva lista vacía.
        public List<Producto>? Productos { get; set; } = new List<Producto>();
    }
}

```

3. Clase LoginRequest.cs

A diferencia de las anteriores, esta no es una entidad que "viva" en una base de datos. Es un **DTO (Data Transfer Object)**, una clase diseñada exclusivamente para capturar las credenciales que el usuario envía desde el formulario de login.

```
namespace API_RESTful.Models
{
    public class LoginRequest
    {
        public string Correo { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
    }
}

```

Capa de Repositorios: Nuestra Bóveda de Datos en Memoria

En la carpeta **Repositories**, crearemos la clase **RepositorioTienda.cs**. Esta clase es la encargada de gestionar las listas donde vivirá la información de nuestros usuarios y productos mientras la aplicación esté corriendo.



```
using API_RESTful.Models;

namespace API_RESTful.Repositories
{
    public class RepositorioTienda
    {
        // Estas listas actúan como nuestras "tablas" de base de datos
        private readonly List<Usuario> _usuarios = new List<Usuario>();
        private readonly List<Producto> _productos = new List<Producto>();

        // Métodos para gestionar Usuarios
        public List<Usuario> ObtenerUsuarios() => _usuarios;

        public void AgregarUsuario(Usuario usuario)
        {
            // Lógica simple para auto-incrementar el ID
            usuario.Id = _usuarios.Count + 1;
            _usuarios.Add(usuario);
        }

        // Métodos para gestionar Productos
        public List<Producto> ObtenerProductos() => _productos;

        public void AgregarProducto(Producto producto)
        {
            producto.Id = _productos.Count + 1;
            _productos.Add(producto);
        }

        public bool ExisteProducto(string nombre)
        {
            return _productos.Any(p => p.Nombre.Equals(nombre, StringComparison.OrdinalIgnoreCase));
        }
    }
}
```

Registro en Program.cs: ¿Por qué usar Singleton?

Ahora que tenemos nuestra clase, debemos "presentársela" a .NET. Para esto, regresamos al archivo **Program.cs**.

Aquí utilizaremos el método **AddSingleton**. Este es un momento clave de la conferencia para explicar el concepto de "Vida Útil" (Lifetime) de los objetos.

¿Qué es un Singleton?
Es un patrón de diseño que garantiza que **solo exista una única copia** de esta clase en toda la memoria del servidor. Todos los usuarios que entren a la API compartirán exactamente el mismo objeto **RepositorioTienda**.

¿Por qué es obligatorio aquí?
Si registráramos el repositorio como "Transient" o "Scoped", .NET crearía una lista nueva y vacía cada vez que alguien hiciera clic en un botón. ¡Perderíamos los datos inmediatamente! Al ser **Singleton**, las listas _usuarios y _productos se mantienen vivas y con datos mientras el servidor no se apague.

Código para Program.cs:
Busca la sección donde se configuran los servicios (antes del **builder.Build())** y agrega esta línea:

```
builder.Services.AddSingleton<API_RESTful.Repositories.RepositorioTienda>();
```



```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
//builder.Services.AddOpenApi();
// Agregamos el generador de Swagger (Reemplaza a AddOpenApi)
builder.Services.AddSwaggerGen();

// Registrar el Repositorio como una instancia única global (Singleton)
builder.Services.AddSingleton<API_RESTful.Repositories.RepositorioTienda>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    //app.MapOpenApi();
    // Activamos la interfaz gráfica de Swagger solo si estamos en modo Desarrollo
    app.UseSwagger();
    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();

```

Puntos clave:

1. **Encapsulamiento:** Las listas son private. Nadie fuera de esta clase puede borrarlas o modificarlas directamente; solo pueden usar los métodos públicos que nosotros definimos.
2. **Persistencia temporal:** Si detienen la depuración en Visual Studio, los datos se borran porque solo existen en la memoria RAM.

3. **Inyección lista:** Al registrarlo en Program.cs, ya dejamos el camino pavimentado para que, en el siguiente paso, el **Servicio** pueda pedir el repositorio en su constructor.

La Capa de Servicios: El Cerebro de la Operación

En la arquitectura de software profesional, existe una regla de oro: **Los Controladores no piensan, solo coordinan**. Toda la lógica real, las matemáticas, las validaciones (como verificar si un correo ya existe antes de crearlo) y las reglas de negocio deben vivir en la carpeta **Services**.

Para que nuestro Servicio de Usuarios (**UsuarioService**) pueda guardar datos, necesita comunicarse con nuestro Repositorio (**RepositorioTienda**). Pero aquí entra un concepto fundamental de .NET: **La Inyección de Dependencias.**

¿Qué es la Inyección de Dependencias?
En lugar de que nuestro Servicio cree el Repositorio manualmente escribiendo **var repo = new RepositorioTienda();**, nosotros simplemente "pedimos" el Repositorio en el constructor de la clase. .NET se encargará de buscar esa instancia en la memoria y entregárnosla automáticamente.

- **La analogía:** Imagina que eres un cirujano (el Servicio) a punto de operar. Tú no fabricas el bisturí en el quirófano; simplemente extiendes la mano y pides "bisturí". El asistente (.NET) te inyecta la herramienta exacta que necesitas en la mano.

Creando nuestro UsuarioService.cs

Dentro de la carpeta **Services**, crearemos la clase **UsuarioService**. Observa detalladamente el constructor:



```
using API_RESTful.Models;
using API_RESTful.Repositories;

namespace API_RESTful.Services
{
    public class UsuarioService
    {
        // Variable privada de solo lectura para guardar nuestro repositorio
        private readonly RepositorioTienda _repositorio;

        // CONSTRUCTOR: ¡Aquí ocurre la Inyección de Dependencias!
        public UsuarioService(RepositorioTienda repositorio)
        {
            _repositorio = repositorio;
        }

        // Validar y crear un usuario
        public void CrearUsuario(Usuario nuevoUsuario)
        {

            // Lógica de negocio: Validar que el correo no exista ya
            var existe = _repositorio.ObtenerUsuarios()
                                     .Any(u => u.Correo == nuevoUsuario.Correo);

            if (existe)
            {
                throw new Exception("Error: El correo ya está registrado.");
            }

            // Logica de negocio 2: Validar que los productos asociados existan en el catálogo global
            if (nuevoUsuario.Productos != null && nuevoUsuario.Productos.Count > 0)
            {
                foreach (var producto in nuevoUsuario.Productos)
                {
                    // Si el producto no existe en el catálogo global, disparamos el error
                    if (!_repositorio.ExisteProducto(producto.Nombre))
                    {
                        throw new Exception($"El producto '{producto.Nombre}' no existe. Por favor, créelo primero en el catálogo.");
                    }
                }
            }

            // Si todo está bien, le decimos al repositorio que lo guarde
            _repositorio.AgregarUsuario(nuevoUsuario);
        }

        // Obtener la lista de usuarios
        public List<Usuario> ObtenerUsuarios() => _repositorio.ObtenerUsuarios();
    }
}

```

El Paso Clave: Avisarle a Program.cs

Si ejecutamos el código justo ahora e intentamos usar este Servicio, la aplicación se va a caer y nos dará un error. ¿Por qué? Porque el cirujano extendió la mano pidiendo el bisturí, ¡pero olvidamos contratar al asistente!

Para que esta "magia" de la inyección de dependencias funcione, debemos registrar tanto nuestro Repositorio como nuestro Servicio en el archivo **Program.cs**. Es allí donde

le diremos a .NET: *"Oye, cuando alguien pida un Repositorio, dale la misma instancia siempre (Singleton). Cuando alguien pida un Servicio, dale uno nuevo (Scoped)"*.

Agrego al **Program.cs** lo siguiente, siempre antes de **var app:**

builder.Services.AddScoped<API_RESTful.Services.UsuarioService>();

¿Por qué Scoped?
Como explicamos en la arquitectura, esto le dice a .NET: "Crea un Servicio NUEVO por cada petición HTTP que llegue". Esto garantiza que si dos usuarios intentan crear un usuario exactamente al mismo tiempo, cada uno tendrá su propio **UsuarioService** trabajando de forma aislada, evitando que sus datos colisionen. Al terminar de responder, el servicio se destruye, manteniendo nuestra memoria RAM limpia.



```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
//builder.Services.AddOpenApi();
// Agregamos el generador de Swagger (Reemplaza a AddOpenApi)
builder.Services.AddSwaggerGen();

// Registrar el Repositorio como una instancia única global (Singleton)
builder.Services.AddSingleton<API_RESTful.Repositories.RepositorioTienda>();

// 2. Registro de los "Cerebros" (Scoped: 1 por cada petición HTTP)
builder.Services.AddScoped<API_RESTful.Services.UsuarioService>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    //app.MapOpenApi();
    // Activamos la interfaz gráfica de Swagger solo si estamos en modo Desarrollo
    app.UseSwagger();
    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();

```

Creando el ProductoService.cs

Dentro de la misma carpeta **Services**, crearemos el servicio encargado de gestionar nuestro catálogo de productos. Al igual que con los usuarios, inyectaremos nuestro Repositorio (la bóveda) a través del constructor.

La diferencia aquí es que demostraremos cómo el Servicio actúa como un "filtro de calidad". Antes de dejar que un producto llegue a la base de datos, el servicio debe validar que la información tenga sentido.

```
using API_RESTful.Models;
using API_RESTful.Repositories;

namespace API_RESTful.Services
{
    public class ProductoService
    {
        private readonly RepositorioTienda _repositorio;

        // Inyección de dependencias
        public ProductoService(RepositorioTienda repositorio)
        {
            _repositorio = repositorio;
        }

        // Método para listar todos los productos
        public List<Producto> ObtenerTodos()
        {
            return _repositorio.ObtenerProductos();
        }

        // Método para crear un nuevo producto con validaciones
        public void AgregarProducto(Producto nuevoProducto)
        {
            // Regla de Negocio 1: El nombre no puede estar vacío
            if (string.IsNullOrWhiteSpace(nuevoProducto.Nombre))
            {
                throw new Exception("Error: El nombre del producto es obligatorio.");
            }

            // Regla de Negocio 2: Lógica de precios
            if (nuevoProducto.Precio <= 0)
            {
                throw new Exception("Error: El precio del producto debe ser mayor a Q0.00.");
            }

            // Regla de Negocio 3: No permitir productos con el mismo nombre
            if (_repositorio.ExisteProducto(nuevoProducto.Nombre))
            {
                throw new Exception($"El producto '{nuevoProducto.Nombre}' ya existe en el catálogo.");
            }

            // Si pasa todas las validaciones (filtros de calidad), lo guardamos
            _repositorio.AgregarProducto(nuevoProducto);
        }
    }
}

```

De igual forma agrego el **ProductoService** al **Program.cs**.

builder.Services.AddScoped<API_RESTful.Services.ProductoService>();

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
//builder.Services.AddOpenApi();
// Agregamos el generador de Swagger (Reemplaza a AddOpenApi)
builder.Services.AddSwaggerGen();

// Registrar el Repositorio como una instancia única global (Singleton)
builder.Services.AddSingleton<API_RESTful.Repositories.RepositorioTienda>();

// 2. Registro de los "Cerebros" (Scoped: 1 por cada petición HTTP)
builder.Services.AddScoped<API_RESTful.Services.UsuarioService>();
builder.Services.AddScoped<API_RESTful.Services.ProductoService>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    //app.MapOpenApi();
    // Activamos la interfaz gráfica de Swagger solo si estamos en modo Desarrollo
    app.UseSwagger();
    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.UseAuthorization();

app.MapControllers();

app.Run();

```

Los Controladores: Exponiendo nuestra API al mundo

Dentro de la carpeta **Controllers** (recuerda que eliminamos el WeatherForecastController de prueba), vamos a crear nuestros propios recepcionistas.

Cada controlador en .NET necesita heredar de la clase **ControllerBase** y llevar unas "etiquetas" (Atributos) especiales en la parte superior para que Swagger sepa que existen y cómo encontrarlos.

1. El Arte del Enrutamiento (Routing) en .NET

La forma en que .NET maneja las URLs de nuestra API es increíblemente inteligente. Vamos a desglosar tres trucos esenciales que todo desarrollador backend debe conocer.

El Comodín [controller]

Cuando observas la parte superior de tus controladores, notarás esta etiqueta: [Route("api/[controller]")]. Ese [controller] entre corchetes no es texto estático, es una variable dinámica. ASP.NET Core busca ese comodín y lo reemplaza automáticamente por el nombre de tu clase, omitiendo la palabra "Controller".

- UsuarioController -> Se convierte en la ruta /api/Usuario
- ProductosController -> Se convierte en la ruta /api/Productos

¿Por qué es una buena práctica? Si mañana refactorizas tu código y decides renombrar la clase a ClientesController, la ruta se actualizará automáticamente a /api/Clientes. ¡Menos trabajo manual y menos margen de error!

El Misterio de la Virgulilla ~ (Tilde)

Normalmente, la URL de un endpoint es una suma matemática: **Ruta del Controlador + Ruta del Método**. Por ejemplo: /api/Usuario + /listar = /api/Usuario/listar.
Sin embargo, a veces necesitamos romper esa regla usando la **Anulación de prefijo de ruta** (Route Override). Si colocas una virgulilla ~ (o una barra /) al inicio de la ruta de un método, le indicas a .NET: *"Ignora la ruta del controlador, quiero que este endpoint empiece desde la raíz absoluta del servidor"*.

El Caso de Uso Real (Pro-Tip): Cuando despliegas una API en la nube (AWS, Azure, Docker), los balanceadores de carga necesitan verificar si tu servidor está vivo. Para esto, te piden crear un endpoint de "salud" (Health Check). No tiene sentido esconder esta ruta bajo /api/Usuario/isalive; lo ideal es que sea global y de acceso rápido: tuservidor.com/isalive.

```
// Al usar la ~, este endpoint se expone en la raíz absoluta
[HttpGet("~/isalive")]
public IActionResult VerificarSalud()
{
    return Ok("El servidor está funcionando correctamente");
}
```

Agrupación en Swagger con [Tags]

Si pruebas el código anterior, notarás un pequeño detalle visual: aunque la ruta es global (**/isalive**), Swagger la seguirá metiendo dentro de la sección "Usuario". Esto ocurre porque Swagger agrupa visualmente basándose en el archivo físico donde escribiste el código.
Para sacar ese método de ahí y darle su propia categoría independiente en la interfaz, simplemente usamos el atributo **[Tags]**:

```
[Tags("Mantenimiento del Sistema")]
[HttpGet("~/isalive")]
public IActionResult VerificarSalud()
{
    // ...
}
```

2. Entendiendo IActionResult y los Codigos de Estado

En el desarrollo web tradicional, el backend solía devolver archivos HTML completos para que el navegador los dibujara. En una API RESTful, el paradigma cambia: nosotros no devolvemos pantallas, **devolvemos datos y un Código de Estado HTTP**.
La magia de usar la interfaz **IActionResult** como tipo de retorno en nuestros métodos es la flexibilidad. Nos permite que una misma función devuelva diferentes respuestas dependiendo de lo que haya ocurrido en la lógica de negocio.

Existen tres grandes familias de respuestas que debemos dominar:

Familia de los 200: "Todo salió bien"

El servidor entendió la petición y la procesó con éxito.

- **return Ok(datos); (200 OK):** El más común. Ideal para responder a un GET entregando la lista de productos solicitada.
- **return Created(url, datos); (201 Created):** La mejor práctica para un POST. Confirma que la información enviada fue insertada exitosamente en la base de datos.
- **return NoContent(); (204 No Content):** Perfecto para un DELETE. Significa: "Hice lo que me pediste (borrar), pero ya no hay datos que devolverte para que los veas".

Familia de los 400: "El Cliente se equivocó"

El servidor está bien, pero la petición que envió el frontend tiene problemas.

- **return BadRequest(mensaje); (400 Bad Request):** Falla de validación. Por ejemplo, intentar guardar un usuario, pero el correo electrónico no tiene el símbolo "@".
- **return NotFound(); (404 Not Found):** Muy común en búsquedas. "Busqué en la base de datos el ID que solicitaste, pero no existe".
- **return Unauthorized(); (401 Unauthorized):** Problemas de identidad. "Esta ruta es privada y no enviaste tu llave (Token)".
- **return Forbid(); (403 Forbidden):** Problemas de permisos. "Sé perfectamente quién eres, pero no tienes el rango necesario para ejecutar esta acción".

Familia de los 500: "El Servidor falló"

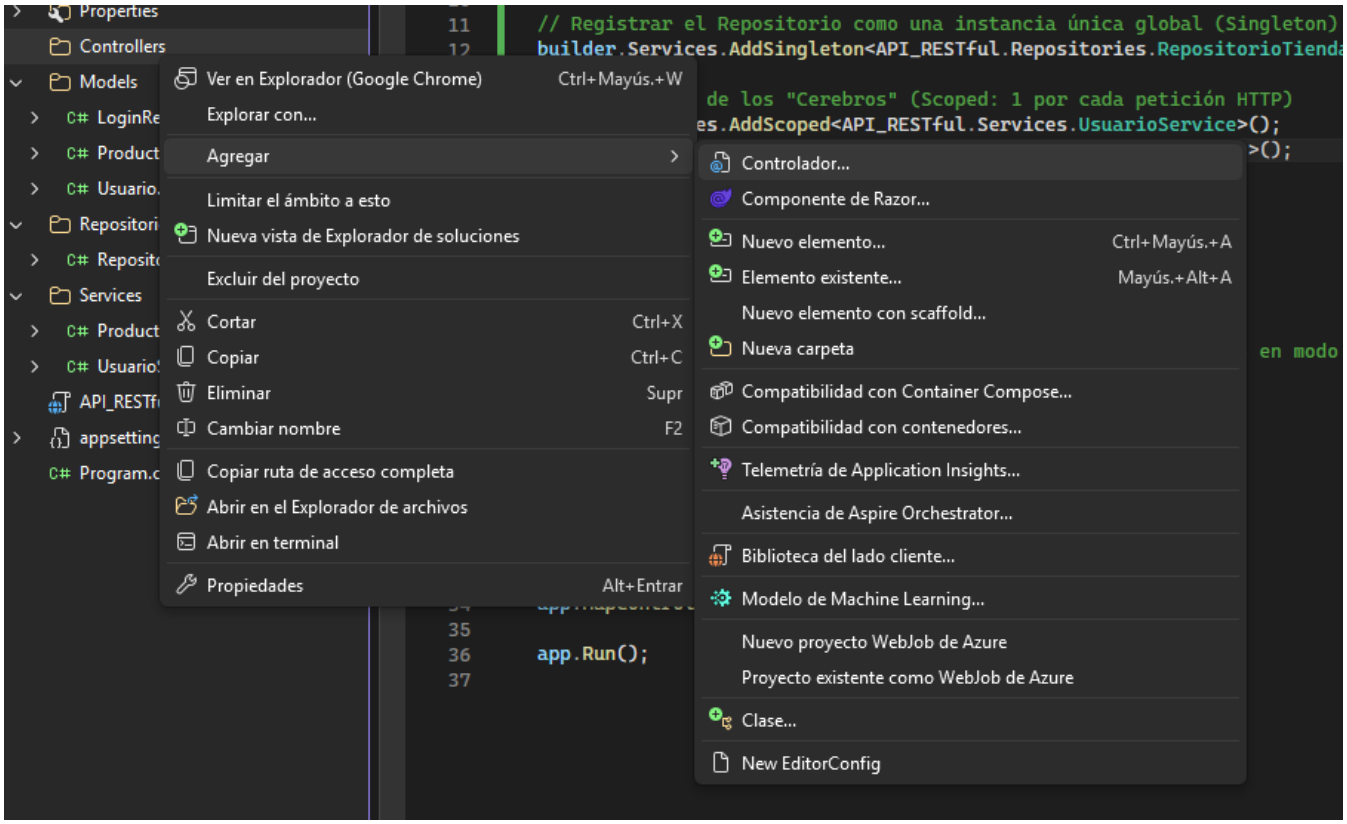
La petición estaba perfecta, pero nuestro código o infraestructura colapsó.

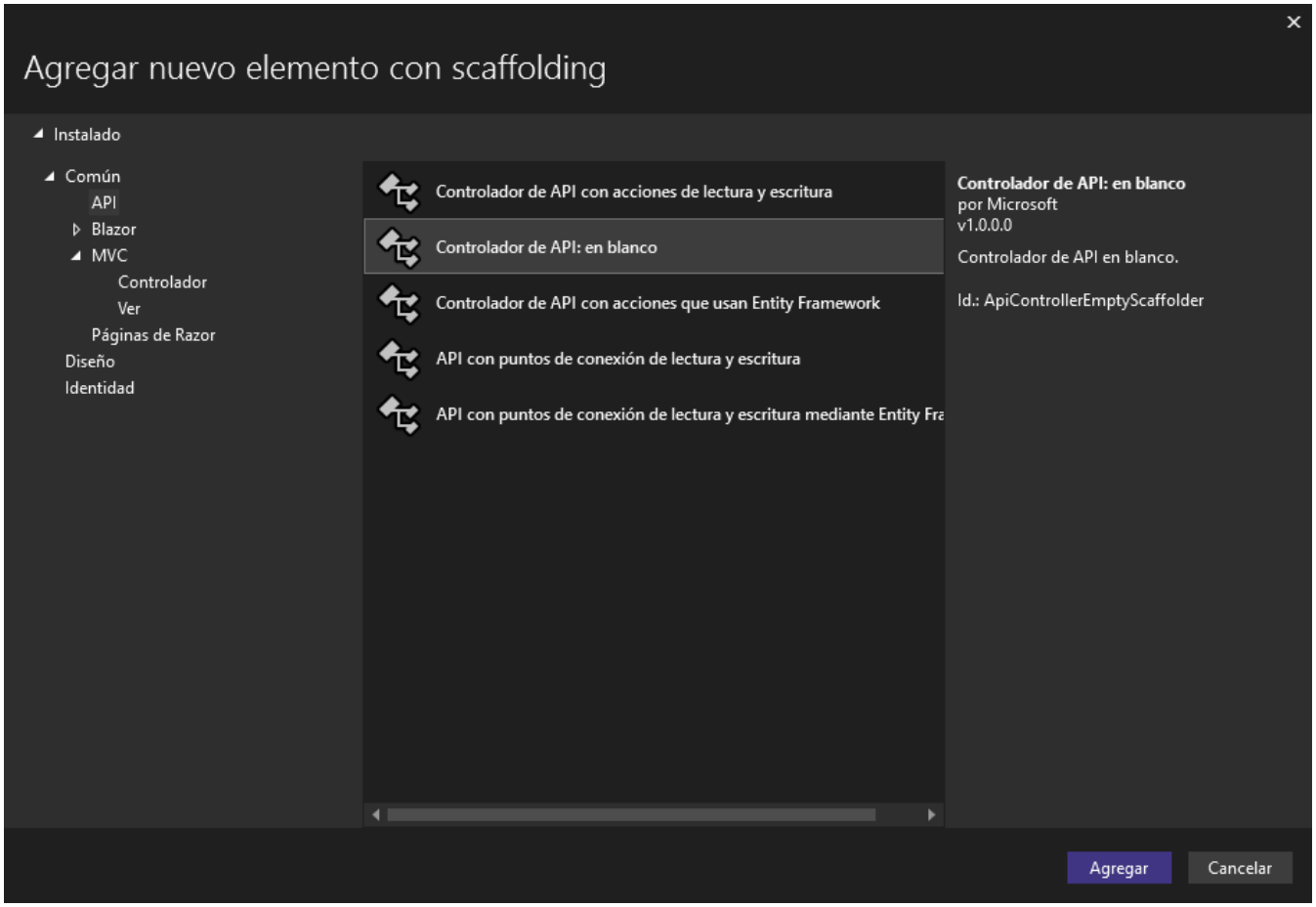
- **return StatusCode(500, "Error crítico"); (500 Internal Server Error):** Indica que se cayó la base de datos, hubo una división por cero o una excepción no controlada en nuestro bloque try/catch.

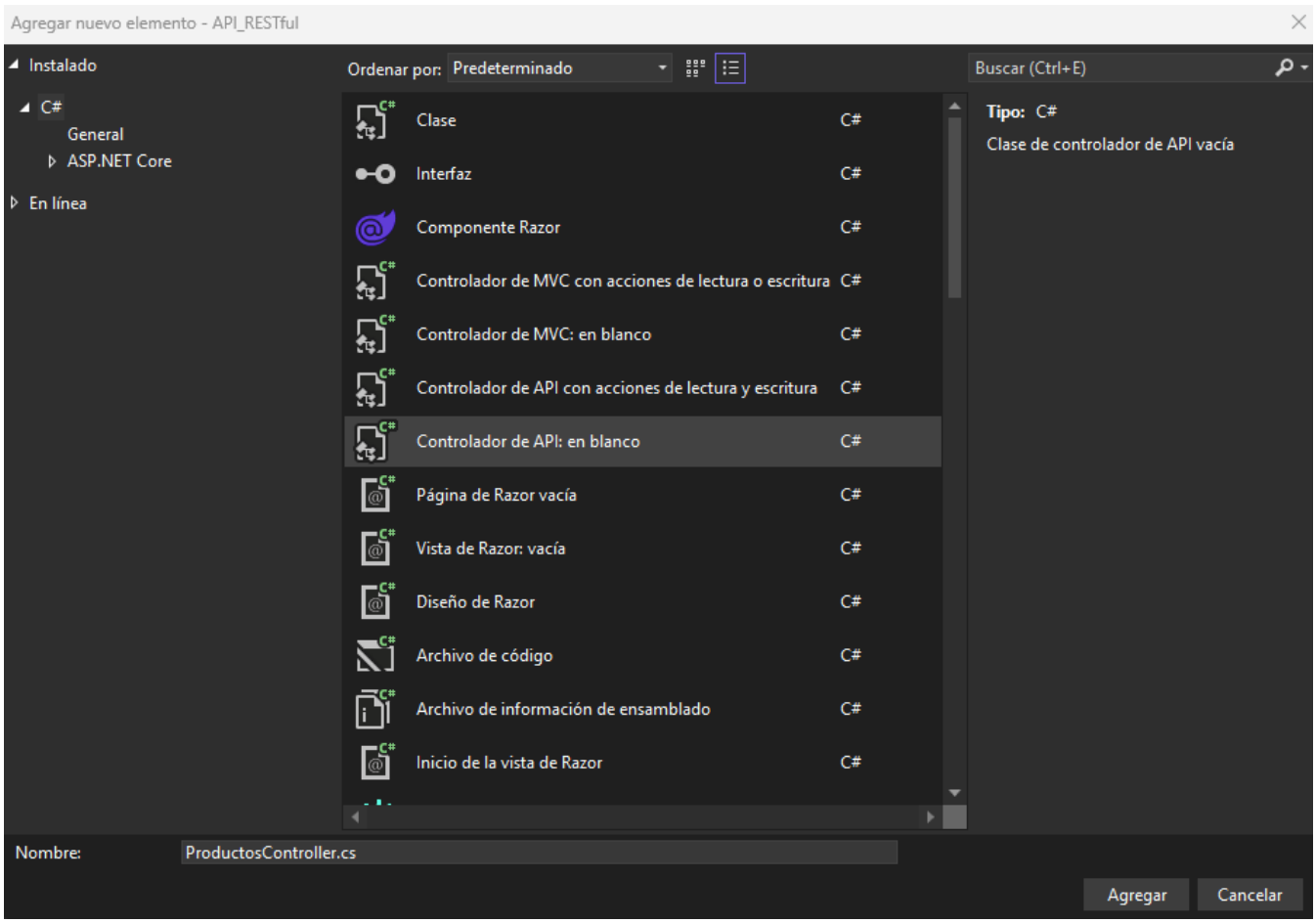
Comprender que no todas las peticiones deben responder con un "200 OK" es el salto de calidad definitivo entre un proyecto escolar básico y un desarrollo de software robusto y profesional.

Creando el ProductosController.cs

Para crear un controlador, dar clic derecho en la carpeta **Controllers**, selecciona **Agregar** > **Controlador**... > **Controlador de API en blanco**. Llámalo ProductosController.




Agregar nuevo elemento con scaffolding
Instalado
Común
API
Blazor
MVC
Controlador
Ver
Páginas de Razor
Diseño
Identidad
Controlador de API con acciones de lectura y escritura
Controlador de API: en blanco
Controlador de API con acciones que usan Entity Framework
API con puntos de conexión de lectura y escritura
API con puntos de conexión de lectura y escritura mediante Entity Fra
Controlador de API: en blanco
por Microsoft
v1.0.0.0
Controlador de API en blanco.
Id.: ApiControllerEmptyScaffolder
Agregar
Cancelar


Agregar nuevo elemento - API_RESTful
Instalado
C#
General
ASP.NET Core
En línea
Ordenar por: Predeterminado
Buscar (Ctrl+E)
Clase C#
Interfaz C#
Componente Razor C#
Controlador de MVC con acciones de lectura o escritura C#
Controlador de MVC: en blanco C#
Controlador de API con acciones de lectura y escritura C#
Controlador de API: en blanco C#
Página de Razor vacía C#
Vista de Razor: vacía C#
Diseño de Razor C#
Archivo de código C#
Archivo de información de ensamblado C#
Inicio de la vista de Razor C#
Tipo: C#
Clase de controlador de API vacía
Nombre: ProductosController.cs
Agregar
Cancelar



```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace API_RESTful.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductosController : ControllerBase
    {
    }
}

```

Acá podemos ver lo que se explico arriba, así se crea el controlador por defecto, nosotros lo conectamos al servicio quedando de la siguiente manera:

Agregamos las rutas en el controlador, se usa la sintaxis explicada arriba, hacemos uso de **[HttpGet], [HttpPost],** también agregamos un **tag** para mostrar como funciona lo separar el endpoint de esta colección de Productos, con la sintaxis **[Tag]**.



```
using API_RESTful.Models;
using API_RESTful.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace API_RESTful.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductosController : ControllerBase
    {
        private readonly ProductoService _servicio;

        // 2. Inyección de Dependencias: Pedimos el Servicio
        public ProductosController(ProductoService servicio)
        {
            _servicio = servicio;
        }

        // 3. Endpoint GET: Para leer datos
        [HttpGet("listar")] // URL: api/Productos/listar
        public IActionResult GetProductos()
        {
            var productos = _servicio.ObtenerTodos();
            return Ok(productos); // Devuelve un HTTP 200 OK con la lista en formato JSON
        }

        // 4. Endpoint POST: Para enviar datos
        [HttpPost("crear")] // URL: api/Productos/crear
        public IActionResult PostProducto([FromBody] Producto nuevoProducto)
        {
            try
            {
                // Le pasamos el producto al Servicio para que aplique las reglas de negocio
                _servicio.AgregarProducto(nuevoProducto);

                return Ok(new { Mensaje = "Producto creado exitosamente" });
            }
            catch (Exception ex)
            {
                // Si el Servicio lanza un error (ej. precio negativo), devolvemos un 400 Bad Request
                return BadRequest(new { Error = ex.Message });
            }
        }

        [HttpGet("~/isalive")]
        // [HttpGet("/isalive")] <-- Esto hace exactamente lo mismo
        [Tags("Monitoreo")] // Esto es solo para organizar mejor la documentación en Swagger, lo separa de esta coleccion de endpoints
        public IActionResult IsAlive()
        {
            return Ok("API está viva y funcionando correctamente.");
        }
    }
}

```

Manejamos los errores que lancemos desde los servicios con try-catch.

Creando el UsuarioController.cs

Repetimos el proceso para los usuarios. Por ahora, crearemos los endpoints básicos de registro y lectura. (Más adelante, en nuestro "Gran Final", agregaremos aquí la ruta de Login y el candado de seguridad).

```
using API_RESTful.Models;
using API_RESTful.Services;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;

namespace API_RESTful.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class UsuariosController : ControllerBase
    {
        private readonly UsuarioService _servicio;

        public UsuariosController(UsuarioService servicio)
        {
            _servicio = servicio;
        }

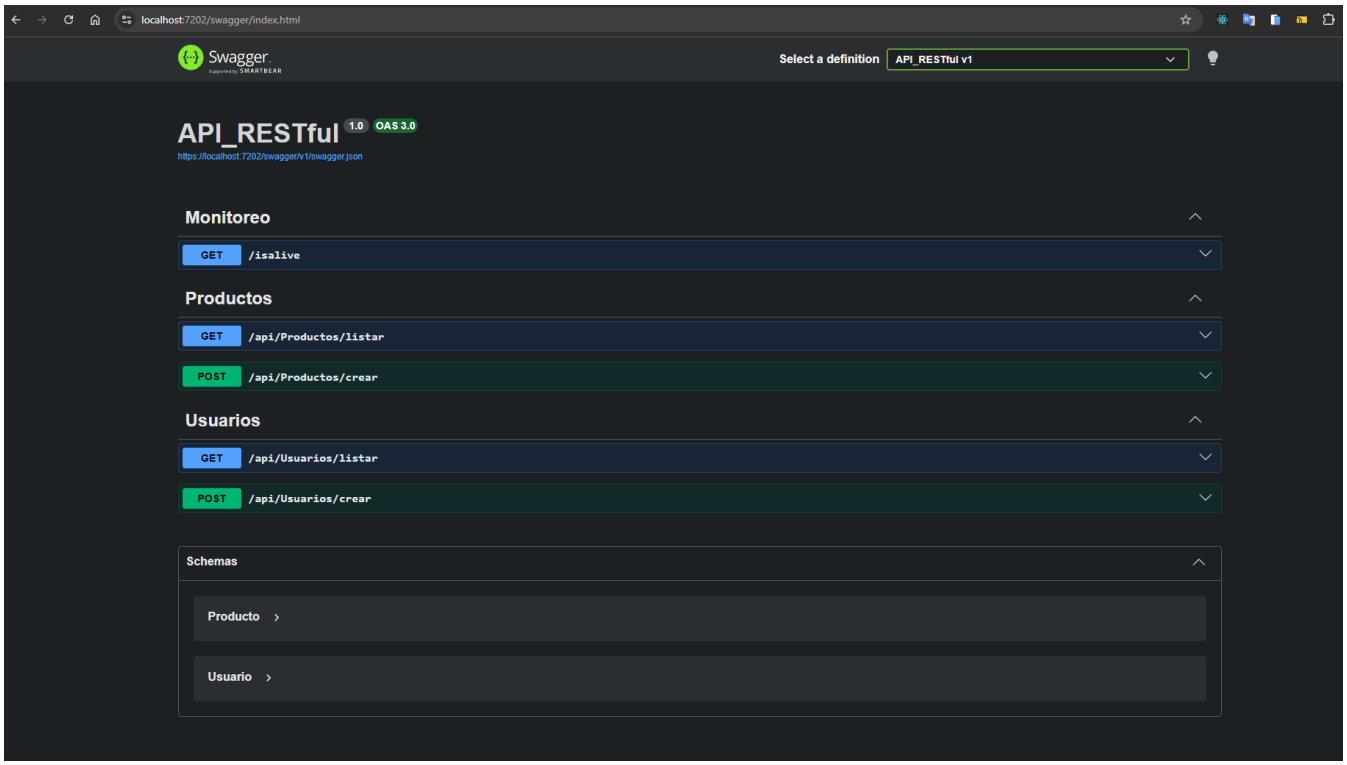
        [HttpGet("listar")]
        public IActionResult GetUsuarios()
        {
            var usuarios = _servicio.ObtenerUsuarios();
            return Ok(usuarios);
        }

        [HttpPost("crear")]
        public IActionResult PostUsuario([FromBody] Usuario nuevoUsuario)
        {
            try
            {
                _servicio.CrearUsuario(nuevoUsuario);
                return Ok(new { Mensaje = "Usuario registrado exitosamente" });
            }
            catch (Exception ex)
            {
                return BadRequest(new { Error = ex.Message });
            }
        }
    }
}

```

Desmitificando Swagger: La Magia de la Reflexión en .NET

Ya terminando todo lo anterior, al ejecutar nuestro proyecto se vera de la siguiente manera:



Se puede observar que se crearon las rutas y se agruparon por controladores, además podemos ver los diferentes Modelos que se usan, si se preguntan como es que se detecto esto y se ve visualmente automáticamente, pues todo eso tiene una explicación.

A primera vista, que Swagger dibuje todas nuestras clases, propiedades y relaciones automáticamente en una página web parece magia negra, pero en realidad es pura ingeniería. Swagger (a través de la librería **Swashbuckle**) detecta tus modelos gracias a una característica fundamental de .NET llamada **Reflexión (Reflection)**.

La Reflexión es la capacidad que tiene el código de C# para "inspeccionarse a sí mismo" y leer su propia estructura mientras la aplicación se está ejecutando.

Así es como ocurre el proceso de escaneo paso a paso:

1. El escaneo de los Controladores

Cuando arranca tu aplicación y se ejecuta el método AddSwaggerGen() en el Program.cs, la librería no escanea todas las carpetas de tu proyecto al azar para ver qué encuentra. Va directamente a buscar, una por una, las clases que tengan la etiqueta [ApiController] en la parte superior.

2. Análisis de métodos y parámetros

Una vez dentro de tu controlador (por ejemplo, **UsuariosController**), Swagger empieza a leer las firmas de tus métodos. Al llegar a tu endpoint de creación: **public IActionResult PostUsuario([FromBody] Usuario usuario**) Swagger hace una pausa y deduce: *"¡Ajá! Para que este método funcione, necesito recibir un objeto del tipo **Usuario**"*.

3. Disección del Modelo (Aquí ocurre la magia)

Swagger toma esa clase **Usuario** y usa Reflexión para leer su estructura interna. Detecta todas las propiedades que marcaste como public y las anota:

- Ve que **Nombre** es un **string** y lo registra.
- Si existiera **Edad** ve que es un **int** y lo registra.
- De repente, llega a la propiedad **Productos** y ve que es una **List<Producto>.**

Como **Producto** es un tipo complejo que no conoce, Swagger hace una pausa en el Usuario, entra a la clase **Producto**, y repite el proceso recursivamente: lee su Nombre, Precio y Reseña.

4. Traducción a JSON Schema

Finalmente, Swagger traduce todos esos tipos estrictos de C# al estándar universal de OpenAPI, conocido como JSON Schema. El motor sabe mapear los tipos perfectamente:

- Un **string** de C# es un string en JSON.
- Un **int** se convierte en un integer.
- Una **List<>** se convierte en un array (un arreglo **[ ]**).

Con toda esa información recolectada y traducida, construye el archivo de contrato final y dibuja la sección de **"Schemas"** que ves en la parte inferior de tu pantalla interactiva.

Dato Curioso (Pro-Tip): Si tú creas un modelo nuevo en la carpeta Models (por ejemplo, Factura.cs), pero **no lo usas** como parámetro de entrada ni como valor de retorno en ningún Controlador, Swagger lo va a ignorar por completo y no aparecerá en la página. ¡La herramienta es tan inteligente que solo documenta lo que realmente se expone al exterior!

JSON para Probar los Endpoints desde Swagger

Crear Productos



Se puede dar en **Try it out** e ir cambiando los valores que se indican en el **JSON** aca se tienen unos JSON listos para copiar y pegar:

Recordar que el id no se coloca ya que es auto incremental, se deja en 0.

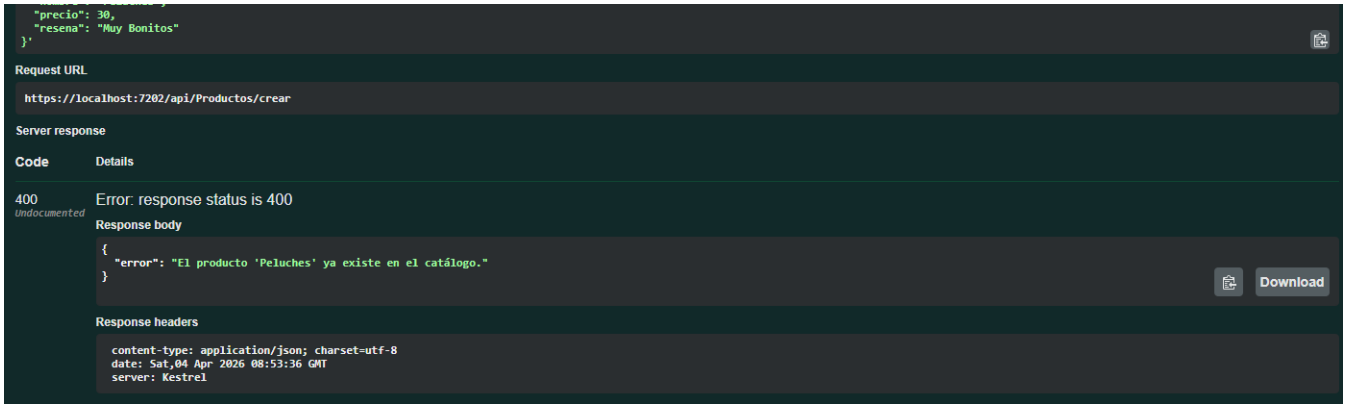
```
{
  "id": 0,
  "nombre": "Peluches",
  "precio": 30,
  "resena": "Muy Bonitos"
}

{
  "id": 0,
  "nombre": "Calcetines",
  "precio": 10,
  "resena": "De buena Calidad"
}

{
  "id": 0,
  "nombre": "Platos",
  "precio": 20,
  "resena": "Buena calidad"
}
```



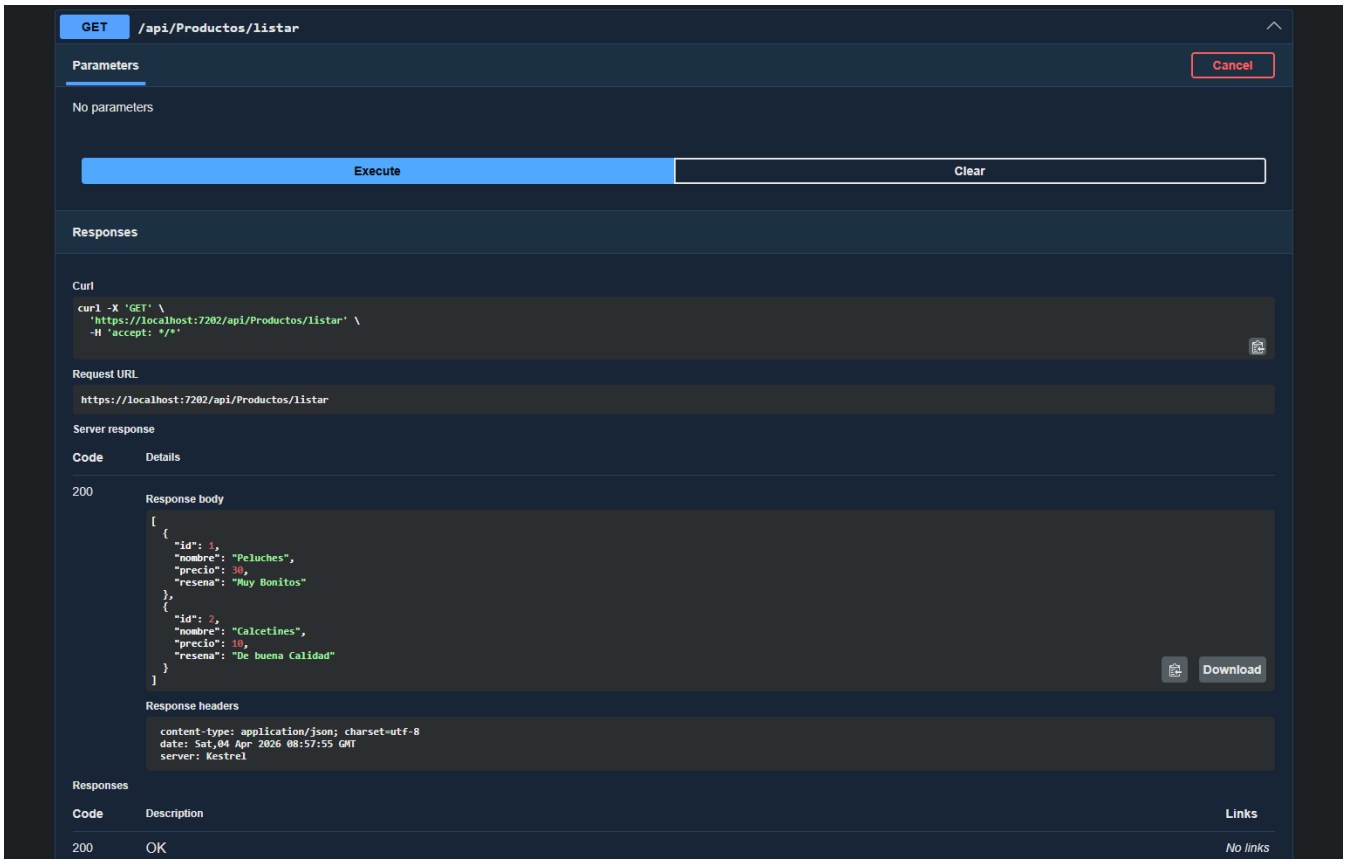

Si todo esta correcto, se retorna un estado **200** indicando que el producto se creo correctamente.



Si se ingresa un nombre repetido de productos, se devuelve un error con código **400**.

Listar Productos

Para este endpoint no se necesita de algún modelo, solo se ejecuta y si existen valores los devuelve como un array de objetos.



Devuelve un código **200** mostrando los productos existentes.

Crear Usuario



Para crear un usuario se necesita lo que indica el modelo, la lista de productos se puede quitar si no se agregaran productos, se pueden copiar y pegar los siguientes **JSON,** de igual forma el id no se agrega ya que es auto incremental.

Usuario sin productos:

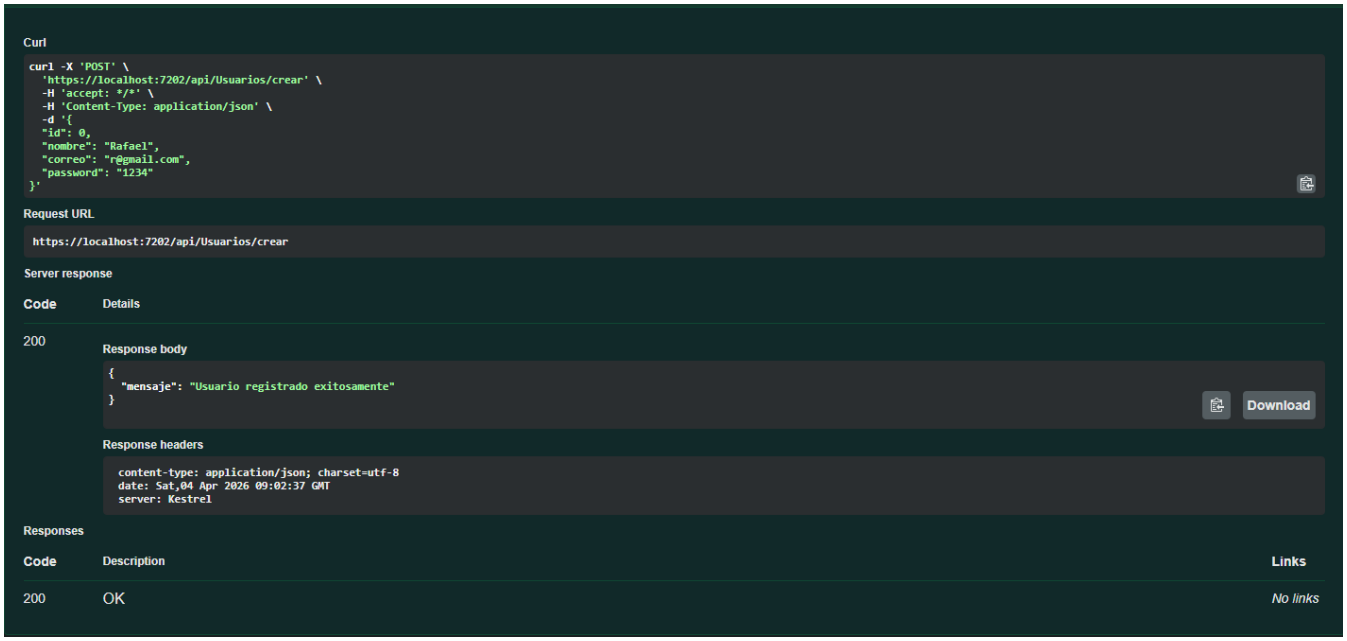
```
{
  "id": 0,
  "nombre": "Rafael",
  "correo": "r@gmail.com",
  "password": "1234"
}
```

Usuario con productos:

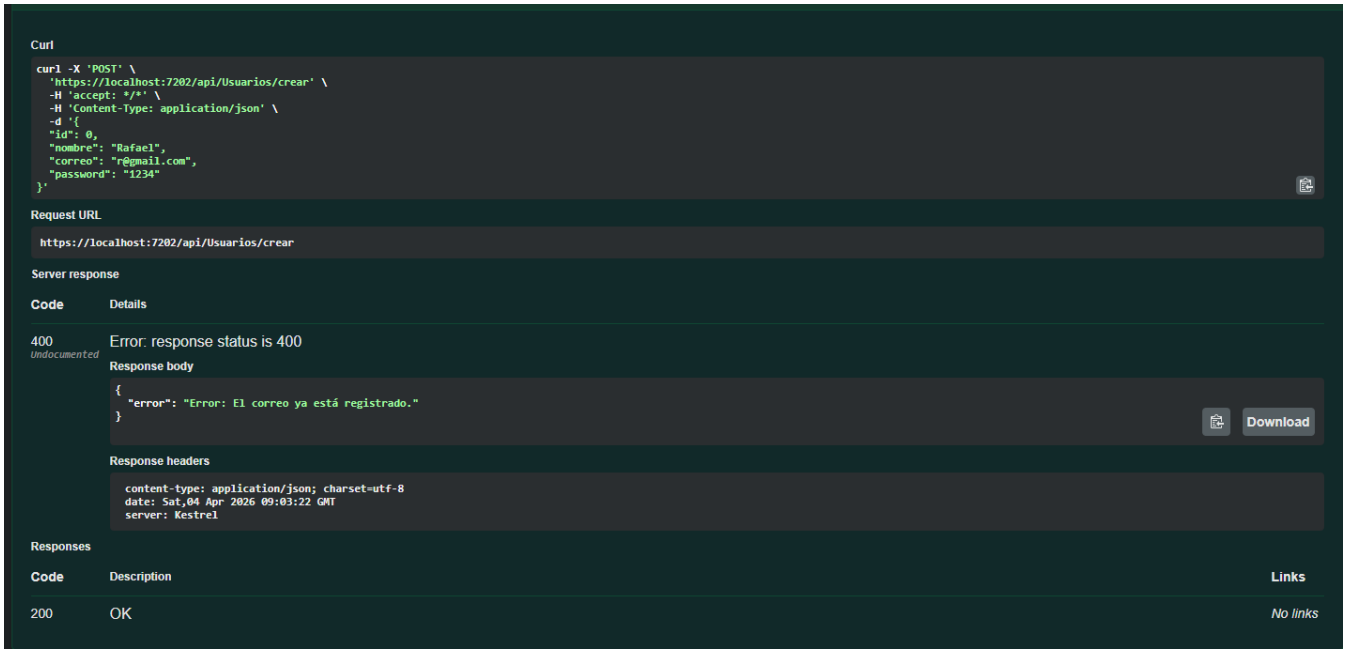
```
{
  "id": 0,
  "nombre": "Tobias",
  "correo": "t@gmail.com",
  "password": "tobias",
  "productos": [
    {
```

```
    "id": 1,
    "nombre": "Peluches",
    "precio": 30,
    "resena": "Muy Bonitos"
   }
  ]
}


{
  "id": 0,
  "nombre": "Miguel",
  "correo": "m@gmail.com",
  "password": "miguel",
  "productos": [
          {
            "id": 1,
            "nombre": "Peluches",
            "precio": 30,
            "resena": "Muy Bonitos"
          },
          {
            "id": 2,
            "nombre": "Calcetines",
            "precio": 10,
            "resena": "De buena Calidad"
          },
          {
            "id": 3,
            "nombre": "Platos",
            "precio": 20,
            "resena": "Buena calidad"
          }
  ]
}
```



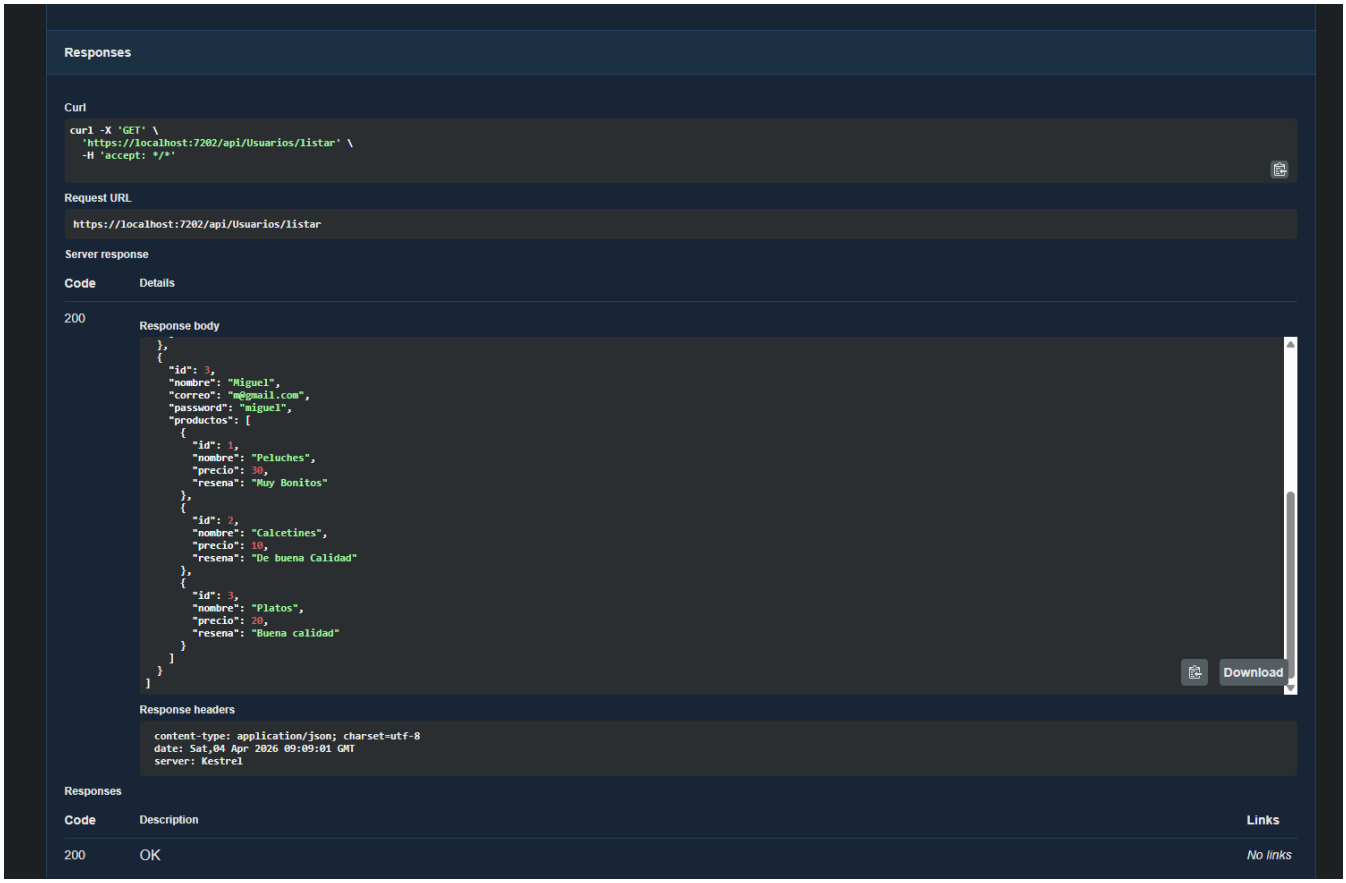
Si todo se ingreso sin errores se retorna un código **200** indicando que el usuario se registro exitosamente.



Si se repite el nombre del usuario se retorna el código **400** indicando que el usuario ya esta registrado.

Listar Usuarios

Para este endpoint no se necesita de algún modelo, solo se ejecuta y si existen valores los devuelve como un array de objetos.



Seguridad con JWT y Pruebas en Postman

Para cerrar con broche de oro nuestra API, vamos a implementar seguridad real. En la industria, no podemos dejar que cualquier persona acceda a todos los datos. Aquí es donde entra **JWT (JSON Web Token).**

¿Qué es JWT?

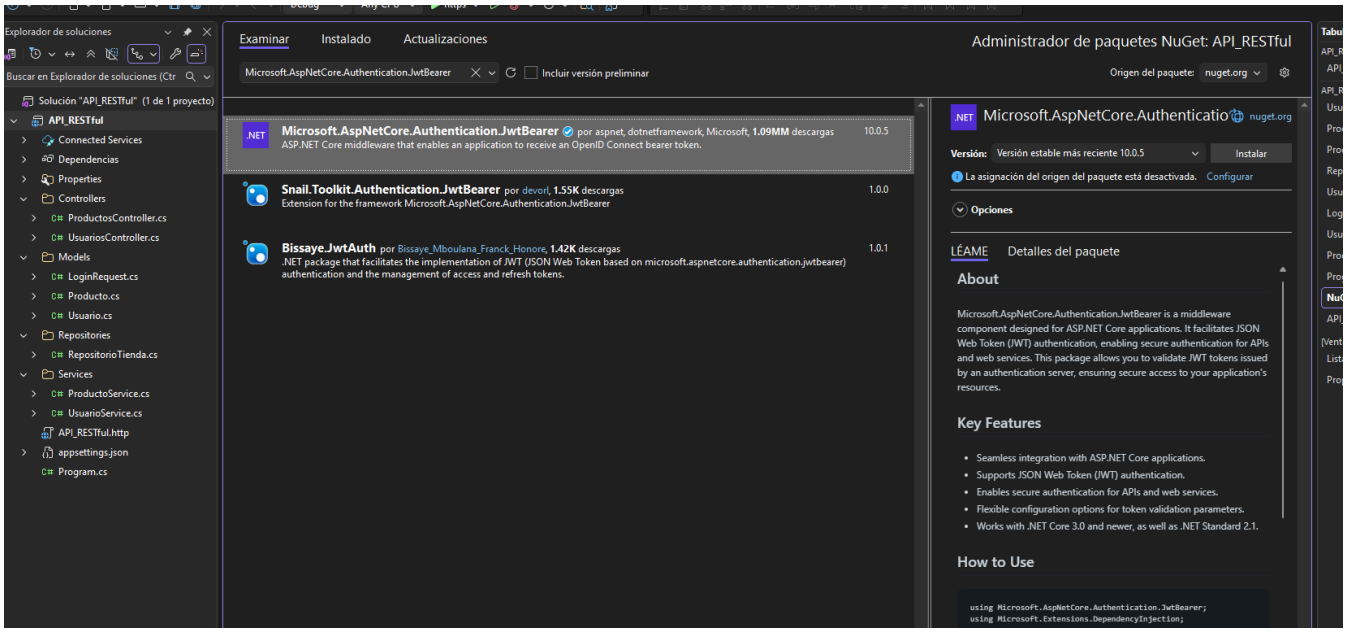
Es un estándar abierto que define una forma compacta y autónoma de transmitir información de forma segura entre las partes como un objeto JSON. En el desarrollo

web, es la llave electrónica que le damos al usuario una vez que inicia sesión correctamente.

Instalación de la Dependencia

Para que .NET entienda cómo generar y leer estos tokens, necesitamos instalar un paquete oficial de Microsoft.

- **Desde Visual Studio (NuGet):** Busca e instala Microsoft.AspNetCore.Authentication.JwtBearer.



- **Desde la Terminal:** Ejecuta **dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer**

1. El Mito de la “Encriptación” y la Clave Secreta

Muchos creemos al principio que el JWT viaja "encriptado" y que la clave sirve para "desencriptarlo" y leer qué hay adentro. **¡Pero esto es una trampa!**

El estándar JWT (sin importar si usas .NET, Node, o Python) por defecto **no encripta los datos, solo los codifica en Base64**. Si tú agarras el token y lo pegas en una página como jwt.io, vas a poder leer perfectamente el correo y el nombre del usuario a simple vista, sin necesidad de ninguna clave.

Entonces, ¿para qué sirve la clave secreta? **Sirve para FIRMAR el token, no para ocultarlo.**

Piensa en el JWT como un cheque bancario de papel:

- Todos pueden leer la cantidad y el nombre escritos en el cheque (el Payload).
- La **Clave Secreta** es la pluma con la que tú (el servidor) firmas ese cheque.
- Si un hacker intenta modificar el cheque (por ejemplo, cambiando su rol de "alumno" a "catedrático"), la firma original se rompe matemáticamente.

¿Por qué usamos la clave en dos lugares distintos del código?

- **En el Servicio (UsuarioService.cs):** Aquí actuamos como el gerente del banco. Usamos la clave secreta para **generar la firma** y estamparla en el token antes de entregárselo al usuario.
- **En Program.cs:** Aquí actuamos como el cajero o el guardia de seguridad. Le decimos a .NET: *"Cada vez que alguien traiga un token a una ruta protegida (**[Authorize]**), usa esta misma clave secreta para **verificar matemáticamente** que la firma es auténtica y que nadie alteró el contenido en el camino"*.

Pro Tip: Poner la clave "quemada" (hardcoded) en el código, como haremos ahora, es solo para el ejemplo didáctico. En un proyecto profesional, esa clave se guarda en un archivo de configuración (**appsettings.json**) o en **variables de entorno**, para que nunca se suba a GitHub.

2. El Ciclo de Vida del Token: Dos Momentos Clave

Momento 1: El Login (Aquí CREAMOS la firma)

Cuando el usuario hace la petición de Login enviando su correo y su password, **todavía no existe ningún token**. Lo que hacemos en ese momento es:

1. Buscar en nuestro Repositorio (nuestra base de datos) si ese correo y esa contraseña existen y coinciden.
2. Si las credenciales son correctas, es cuando fabricamos el token desde cero.
3. Usamos la Clave Secreta para **ponerle nuestra firma** a ese token recién creado y se lo entregamos al usuario.

Analogía: Es como cuando vas al mostrador del hotel. Entregas tu DPI o Pasaporte (correo y contraseña). El recepcionista verifica que tengas una reserva y, si todo está bien, programa una tarjeta electrónica (crea el token) y te la entrega.

Momento 2: Usar las rutas protegidas (Aquí VERIFICAMOS la firma)
La verificación de la firma ocurre **después** del login, cuando el usuario intenta entrar a una ruta que tiene el candado ([Authorize]). Ahí el usuario envía el token en la cabecera "Bearer". Lo único que hace .NET es:

1. Recibir el token.
2. Usar la Clave Secreta que pusiste en el Program.cs para aplicar una fórmula matemática.
3. **Verificar que la firma sea auténtica** y que nadie haya alterado el contenido.
4. Si la firma es correcta y el token no ha expirado, deja pasar la petición.

Analogía: Ya tienes tu tarjeta y vas a tu habitación. Pasas la tarjeta por la cerradura de la puerta. A la puerta no le importa tu DPI, solo lee la tarjeta, verifica su firma magnética y si es válida, te deja entrar.

¡En el Login validas **identidad** (credenciales) y en las rutas protegidas validas el **token** (la firma)!

3. ¿Qué se debe (y qué no se debe) poner en el Payload?

El Payload (el cuerpo del token) debe ser lo más ligero y público posible. Solo debe contener la información estrictamente necesaria para identificar quién hace la petición y qué permisos tiene.

Lo que NUNCA debes poner (Datos Sensibles):

- ❌ Contraseñas (ni siquiera encriptadas).
- ❌ Números de cuenta bancaria o tarjetas.
- ❌ DPI, números de seguro social o teléfonos privados.
- ❌ Direcciones físicas.

Lo que SÍ es seguro y recomendado poner (Datos de Reclamación / Claims):

- ☑ **El ID del usuario (Subject/Sub):** Es el dato más importante. Usualmente es un número entero (**145**) o un identificador único (UUID). Así, cuando la API necesita hacer algo complejo, lee el ID del token y va a buscar a la base de datos de forma segura.
- ☑ **Roles o Permisos:** Cadenas de texto como **"Rol": "Administrador".** Esto es súper útil porque la API puede saber si dejarte borrar un producto o no, sin tener que hacer una consulta a la base de datos en cada clic.
- ☑ **Correos o Nombres de Usuario:** Son datos semi-públicos que sirven para que el Frontend pueda leer el token y mostrar un mensaje de bienvenida sin tener que pedirle esos datos a la API de nuevo.
- ☑ **Fecha de expiración (Exp):** Este lo pone .NET automáticamente, para que el token deje de funcionar después de un tiempo.

En Resumen: *"El JWT es tu pasaporte. Tu pasaporte dice tu nombre, tu foto y tu país (información suficiente para dejarte entrar), pero no lleva impreso el PIN de tu tarjeta de débito ni las llaves de tu casa".*

4. ¿Por qué usamos Postman y qué es un “Barere Token”?

Aunque Swagger es excelente para documentar, **Postman** es la herramienta reina en la industria para realizar pruebas robustas, simular flujos de autenticación complejos y manipular cabeceras HTTP tal como lo haría un Frontend real.

En Postman, al querer entrar a una ruta protegida, seleccionamos el tipo de autorización **"Bearer Token"**.

La palabra "Bearer" en inglés significa literalmente **"Portador"**. Funciona exactamente igual que la tarjeta electrónica del cuarto de hotel. Al hotel no le importa quién eres en el momento de abrir la puerta. Si tú eres el "portador" (el que trae la tarjeta en la mano), la puerta se abre. Por eso en la petición, Postman envía la palabra **Bearer** seguida del token, diciéndole al servidor: *"Oye, déjame pasar, yo soy el portador de esta llave válida"*.

¿Y por qué hay tantas opciones en Postman? La captura de Postman muestra la evolución de la seguridad en la web:

- **Basic Auth:** La forma más antigua. Mandas tu usuario:contraseña pegados y codificados. Poco seguro sin HTTPS estricto.
- **API Key:** Es como un JWT pero "tonto". Una cadena de texto estática (ej. x-api-key: 12345) que no caduca. Ideal para comunicación de servidor a servidor.
- **JWT Bearer:** El estándar actual para aplicaciones web modernas y móviles.
- **OAuth 2.0:** El peso pesado. Es el protocolo que ves cuando una página te dice *"Iniciar sesión con Google"*. Un intermediario garantiza quién eres sin darle tu contraseña a aplicaciones de terceros.

5. Modificaciones en el Código

Para implementar esta seguridad, modificaremos tres archivos en nuestro proyecto.

Paso A: Configurar Program.cs

Limpiamos nuestro archivo base e indicamos que usaremos autenticación por JWT. Agrega los **using** necesarios arriba y la configuración del token antes del **builder.Build()**:

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```

```
// ---> MUY IMPORTANTE: Orden de los Middlewares <---
app.UseAuthentication(); // 1. ¿Quién eres? (Revisa el token)
app.UseAuthorization();  // 2. ¿Puedes pasar? (Revisa si la ruta tiene candado)

app.MapControllers();
app.Run();
```

```
// ---> CONFIGURACIÓN DEL TOKEN JWT <---
var claveSecreta = Encoding.ASCII.GetBytes("EstaEsUnaClaveSuperSecretaParaMiConf

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opciones =>
    {
        opciones.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(claveSecreta),
            ValidateIssuer = false,
            ValidateAudience = false
        };
    });
```

Nuestro archivo

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

builder.Services.AddControllers();
// Learn more about configuring OpenAPI at https://aka.ms/aspnet/openapi
//builder.Services.AddOpenApi();
// Agregamos el generador de Swagger (Reemplaza a AddOpenApi)
builder.Services.AddSwaggerGen();

// ---> CONFIGURACIÓN DEL TOKEN JWT <---
var claveSecreta = Encoding.ASCII.GetBytes("EstaEsUnaClaveSuperSecretaParaMiConferenciaIPC2!!");
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(opciones =>
    {
        opciones.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(claveSecreta),
            ValidateIssuer = false,
            ValidateAudience = false
        };
    });

// Registrar el Repositorio como una instancia única global (Singleton)
builder.Services.AddSingleton<API_RESTful.Repositories.RepositorioTienda>();

// 2. Registro de los "Cerebros" (Scoped: 1 por cada petición HTTP)
builder.Services.AddScoped<API_RESTful.Services.UsuarioService>();
builder.Services.AddScoped<API_RESTful.Services.ProductoService>();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    //app.MapOpenApi();
    // Activamos la interfaz gráfica de Swagger solo si estamos en modo Desarrollo
    app.UseSwagger();
    app.UseSwaggerUI();

}

// ---> MUY IMPORTANTE: Orden de los Middlewares <---
app.UseAuthentication(); // 1. ¿Quién eres? (Revisa el token)

app.UseHttpsRedirection();

app.UseAuthorization();  // 2. ¿Puedes pasar? (Revisa si la ruta tiene candado)

app.MapControllers();

app.Run();

```

Paso B: Actualizar UsuarioService.cs

Añadimos la lógica para buscar al usuario y firmar el cheque (el JWT). Agrega estos **using** en la parte superior:

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```

Y agregaremos este método dentro de la clase:

```
// Metodo para iniciar sesión y obtener un JWT
public string IniciarSesion(string correo, string password)
{
    // 1. Buscamos al usuario en la base de datos (Repositorio)
    var usuario = _repositorio.ObtenerUsuarios()
        .FirstOrDefault(u => u.Correo == correo && u.Password == password);

    if (usuario == null)
    {
        throw new Exception("Error: Credenciales incorrectas.");
    }

    // 2. Si existe, fabricamos y firmamos el JWT
    var tokenHandler = new JwtSecurityTokenHandler();
    var claveSecreta = Encoding.ASCII.GetBytes("EstaEsUnaClaveSuperSecretaParaMiConferenciaIPC2!!");

    // Aquí guardamos los datos públicos (Claims) DENTRO del Payload
    var tokenDescriptor = new SecurityTokenDescriptor
    {
        Subject = new ClaimsIdentity(new[]
        {
    new Claim(ClaimTypes.Email, usuario.Correo),
    new Claim(ClaimTypes.Name, usuario.Nombre)
}),
        Expires = DateTime.UtcNow.AddHours(1), // El pasaporte caduca en 1 hora
        SigningCredentials = new SigningCredentials(new SymmetricSecurityKey(claveSecreta), SecurityAlgorithms.HmacSha256Signature)
    };

    var token = tokenHandler.CreateToken(tokenDescriptor);
    return tokenHandler.WriteToken(token); // Retorna el token fabricado como un string
}
```

Paso C: Actualizar UsuariosController.cs

Por último, exponemos la ruta de **/login** para entregar la llave, y creamos una ruta **/mi-perfil** con el candado **[Authorize]** para verificar que nadie sin llave pueda entrar.

Agrego este **using** arriba:

```
using Microsoft.AspNetCore.Authorization; // Agrega esto arriba
```

```
[HttpPost("login")]
public IActionResult Login([FromBody] LoginRequest request)
{
    try
    {
        var token = _servicio.IniciarSesion(request.Correo, request.Password);
        // Devolvemos el JWT al frontend
        return Ok(new { Token = token, Mensaje = "Login exitoso" });
    }
    catch (Exception ex)
    {
        return Unauthorized(new { Error = ex.Message }); // HTTP 401
    }
}

// ---> ESTE ES EL ENDPOINT PROTEGIDO <---
[Authorize] // ¡El candado! Sin token, Postman recibe un 401 Unauthorized
[HttpGet("mi-perfil")]
public IActionResult VerMiPerfil()
{
    // .NET extrae mágicamente los datos del token válido y los pone en "User"
    var correoUsuario = User.FindFirstValue(ClaimTypes.Email);
    var nombreUsuario = User.FindFirstValue(ClaimTypes.Name);

    return Ok(new
    {
        Mensaje = "¡Lograste entrar a la zona segura!",
        Datos = $"Hola {nombreUsuario}, tu correo es {correoUsuario}"
    });
}
```

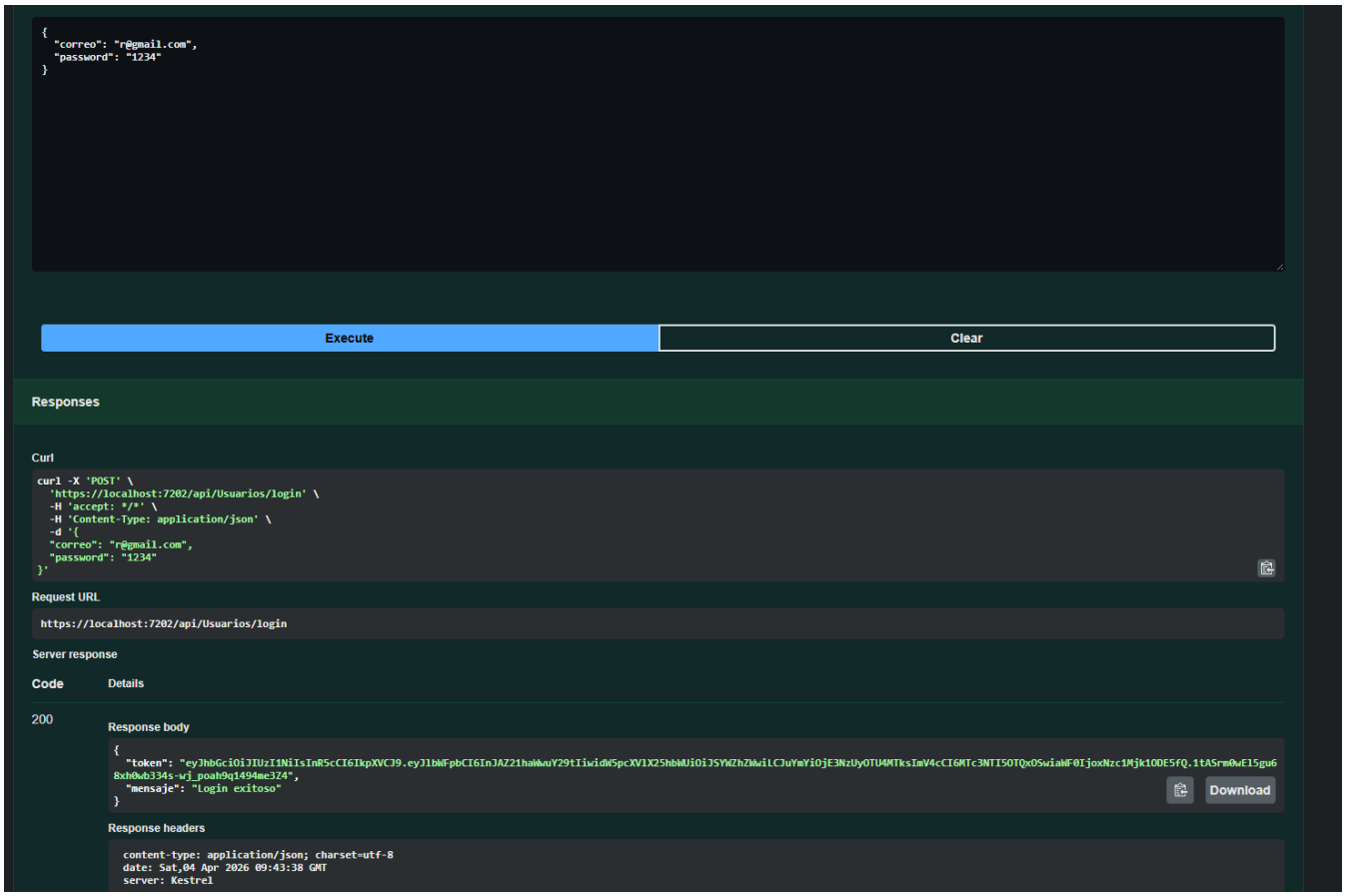
Agrego estos dos métodos en **UsuariosController**.

Paso D: Probar los enpoints en Postman



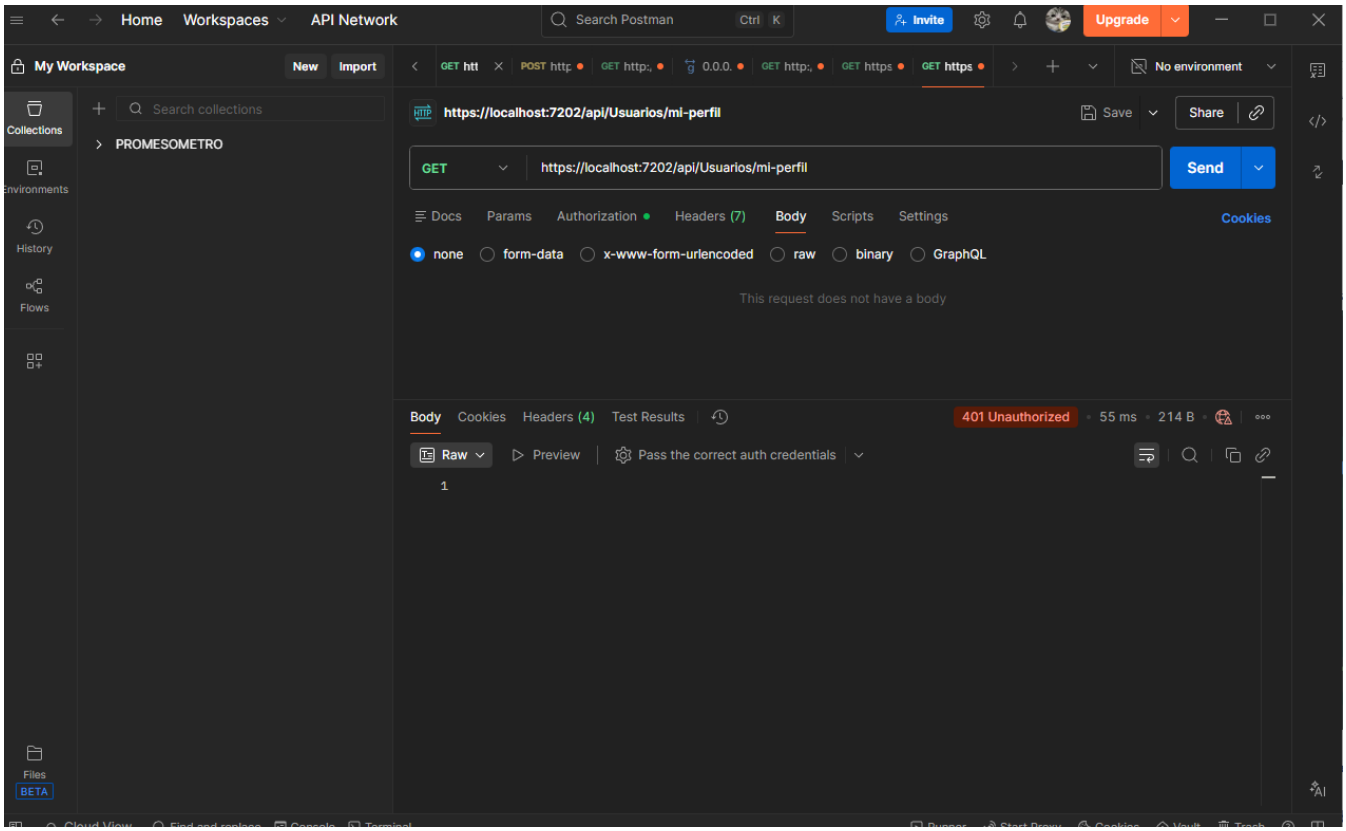
Los enpoints ya aparecen en **Swagger** pero no lo configuramos para que se muestre esa opción ya que probaremos **Postman**.

Hacemos el Login para un usuario existente:

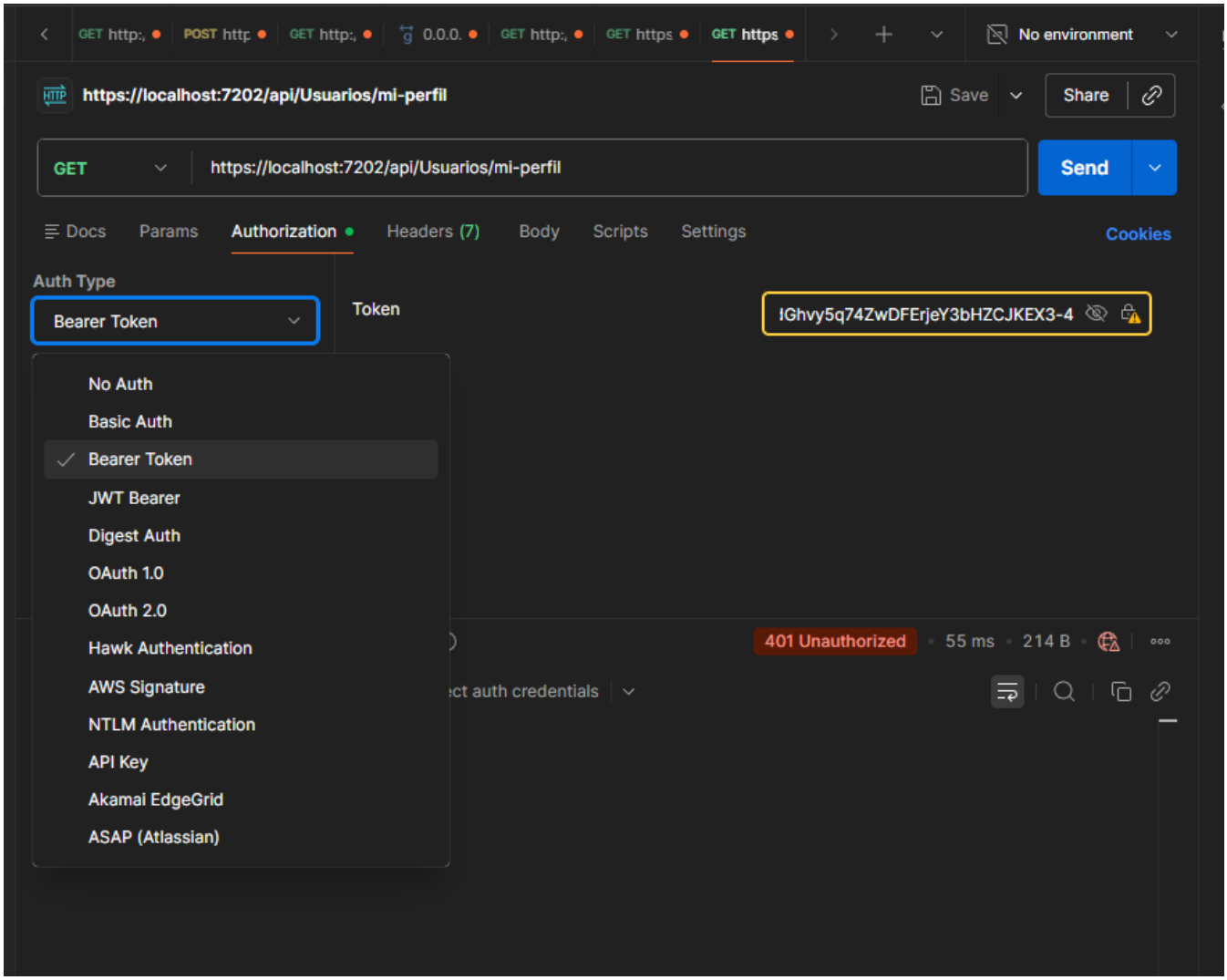


Nos retorno el **token**, si el usuario no existiera retornara un **código de error**.

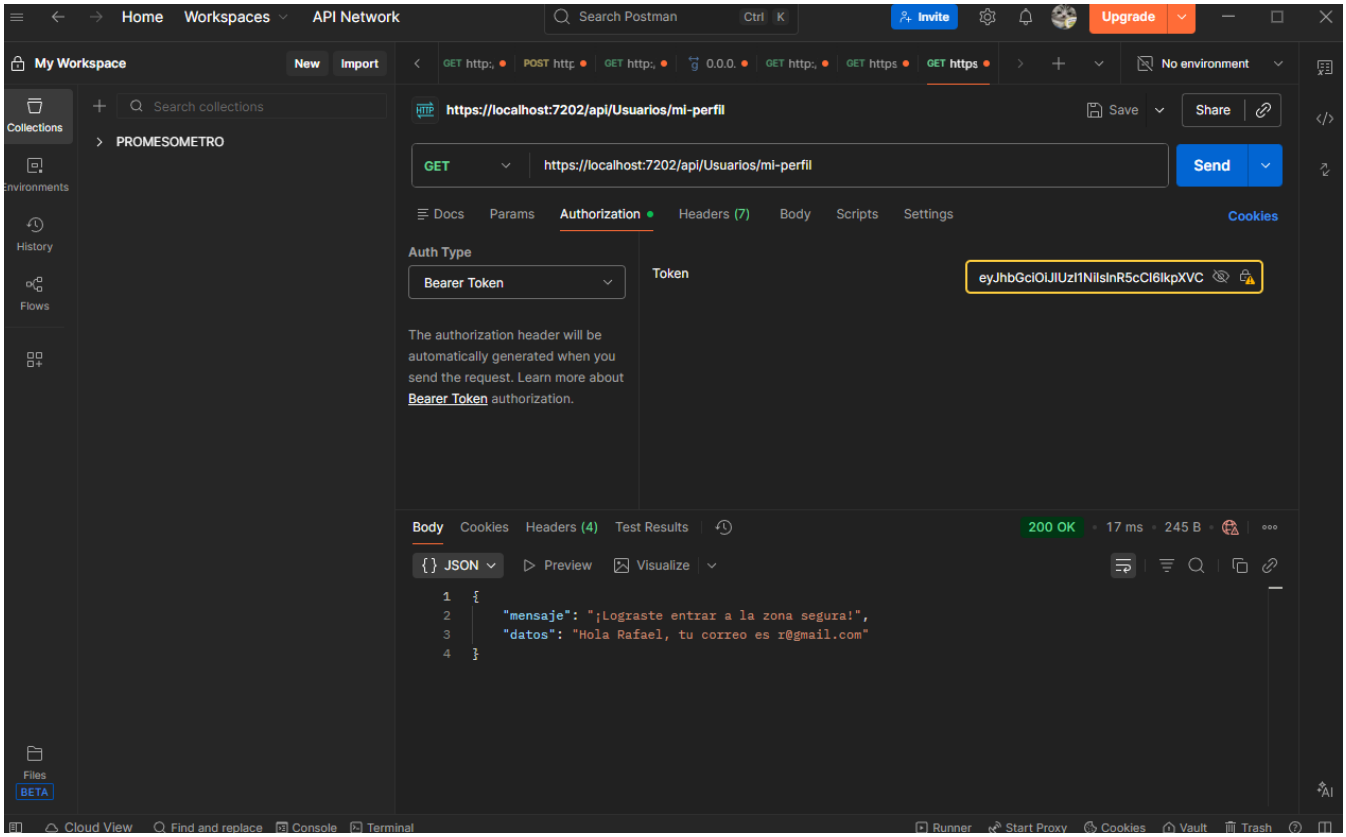
Abrimos **Postman** y colocamos la **url del endpoint**:



Acá nos indica que no estamos autorizados debido a que no mandamos el token.



En esta sección, que es la de **Authorization**, selecionamos **Bearer Token** y pegamos el token en el apartado en amarillo de la derecha.



De esta forma al volver a intentar la petición ya nos devolverá una respuesta, en este caso para el usuario Rafael, de esta manera podemos **proteger las rutas en nuestra API**.